

UNIVERSIDAD DEL CENTRO EDUCATIVO LATINOAMERICANO



Trabajo Práctico

Sistema de Gestión de Inscripciones Académicas

Informe Técnico y Manual de Pantallas

Alumno: González Janin, Manuel

Carrera: ISI – Ingeniería en Sistemas de Información

Materia: Programación Avanzada

Docente: Bressano, Mario

Año: 2026

Índice

1. Abstract	3
2. Manual de pantallas	4
1. Login	4
2. Home Admin	5
3. Listar carreras	6
4. Crear carrera	8
5. Listar cursos	9
6. Crear curso	11
7. Asignar docente	12
8. Listar comisiones	13
9. Alumnos de comisión	24
10. Crear examen	15
11. Listar docentes	16
12. Crear docente	17
13. Cursos del docente	18
14. Listar alumnos	19
15. Crear alumno	20
16. Listar exámenes (admin) – estado pendiente	21
17. Home Docente	22
18. Listar cursos (docente)	23
19. Alumnos de comisión – cierre de cursado	24
20. Home Alumno	25
21. Inscribir carrera	26
22. Inscribir curso	27
23. Inscribir examen	29
24. Historia académica (estado actual)	31
25. Mis exámenes (estado actual)	32
26. Listar exámenes (docente)	33
27. Cargar asistencia	34
28. Listar exámenes (docente) – luego de asistencia	35
29. Cargar notas	36
30. Listar exámenes (docente) – luego de notas	37
31. Ver resultado	38
32. Listar exámenes (admin) – estado cerrado	39
33. Alumnos del examen (admin)	40
34. Historia académica (actualizada)	41
35. Mis exámenes (actualizado)	42
36. Popup de confirmación – baja de carrera	43
Catálogo de acciones con popup de confirmación	44
3. Diagrama Entidad-Relación (DER)	45

1. Abstract

El sistema es una aplicación web Java construida con Servlets y JSP (Jakarta EE, desplegada sobre Apache Tomcat) y persistencia en MySQL mediante JDBC (base de datos “inscripciones”). Su objetivo es gestionar el ciclo completo de inscripciones académicas de una institución, organizado en torno a tres roles: ADMIN, DOCENTE y ALUMNO. El acceso se realiza por email y contraseña; la autenticación crea una sesión con el DNI, el nombre y el rol del usuario, y cada servlet valida ese rol antes de operar (clase utilitaria ServletUtils).

El administrador concentra la gestión maestra: alta, baja lógica (campo Activo) y eliminación —con validación de integridad referencial— de carreras, cursos (que se crean siempre con dos comisiones), comisiones, exámenes, docentes y alumnos. También asigna o quita el docente de cada curso. La eliminación está protegida por reglas de negocio (por ejemplo, no se puede borrar una carrera con cursos, un curso con docente o alumnos, ni un examen con inscriptos o que no esté pendiente).

El docente gestiona sus cursos y exámenes. Sobre cada comisión realiza el cierre de cursado, asignando a cada alumno una condición final (libre, regular o promocionado) y, para los promocionados, una nota entre 8 y 10. Sobre cada examen carga primero la asistencia (presente/ausente) y luego las notas (1 a 10) de los presentes. Estas operaciones se ejecutan dentro de transacciones y disparan cambios de estado encadenados.

El alumno se inscribe a una única carrera, a los cursos de esa carrera (con asignación automática de la primera comisión con cupo) y a los exámenes para los que está habilitado (regular o libre, con más de 24 horas de anticipación). Además consulta su historia académica y sus exámenes inscriptos, y puede darse de baja de la carrera o de un examen (esta última solo con más de 24 horas de antelación).

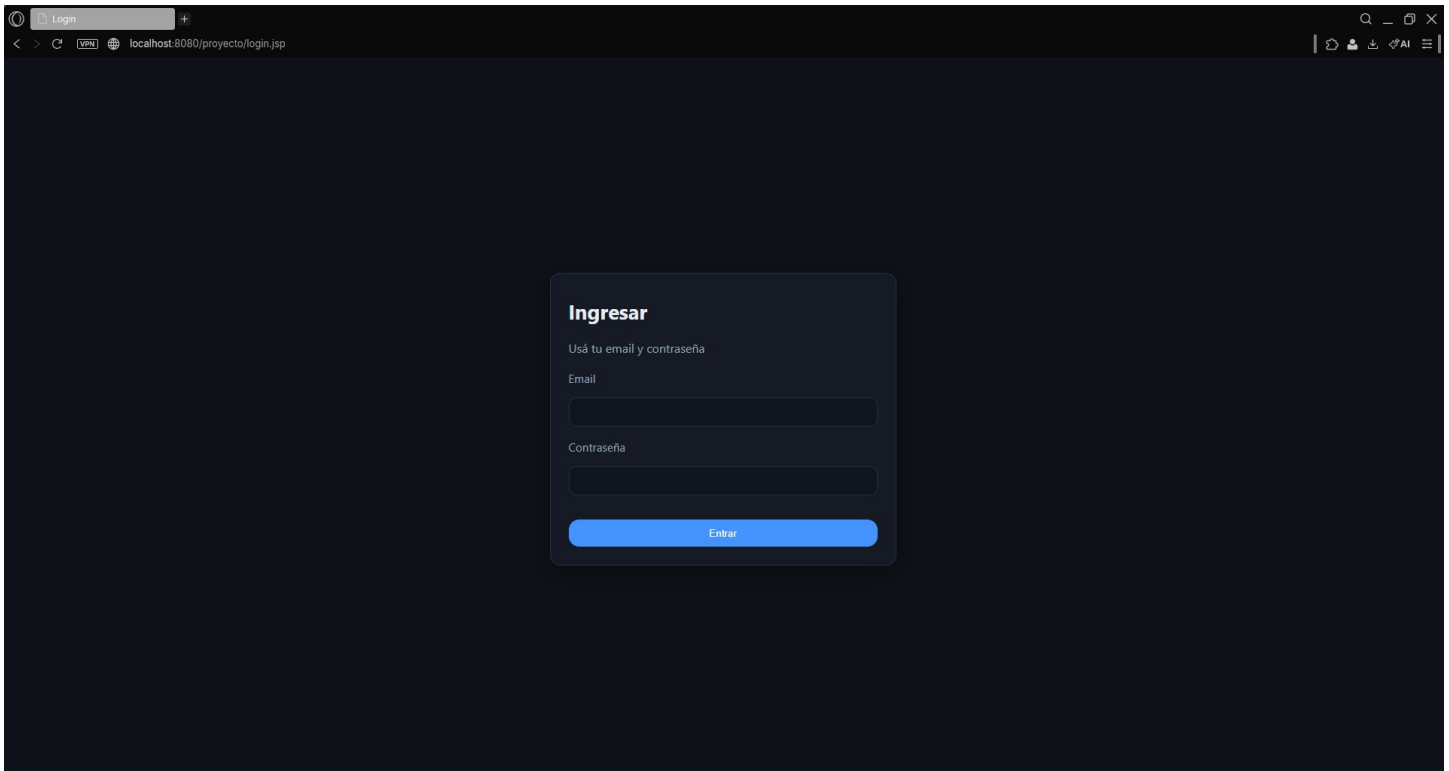
El flujo de estados es central en el sistema. El examen recorre el ciclo Pendiente → Rendido (al cargar asistencia) → Cerrado (al cargar notas), y puede pasar a Cancelado automáticamente. El cursado del alumno evoluciona de “cursando” a regular/libre/promocionado y, al aprobar el examen (nota ≥ 6) o promocionar, a “aprobado”; cuando el alumno aprueba todos los cursos de la carrera, su inscripción a la carrera pasa a APROBADO. Un proceso programado en segundo plano (ExamenListener, ejecutado cada hora) cancela los exámenes pendientes sin inscriptos o sin asistencia presente luego de 72 horas, y marca como ausentes a los alumnos cuya asistencia no fue cargada en ese plazo.

En la capa de presentación, la interfaz utiliza un diseño oscuro propio (CSS) y popovers HTML nativos para las confirmaciones (altas, bajas, habilitaciones y cierres), evitando el uso de JavaScript. Los mensajes de éxito y error se transmiten mediante atributos de request o parámetros de la URL y se muestran como alertas en cada pantalla.

2. Manual de pantallas

A continuación se documentan las pantallas del sistema en el orden del recorrido funcional. Para cada una se indica su descripción, los botones y sus redirecciones, los cambios de estado que genera, las reglas de negocio, las validaciones con el mensaje exacto de cada error y los mensajes de éxito (OK) que puede mostrar.

1. Login



Servlet / JSP: login.jsp (formulario) + LoginServlet (/LoginServlet)

Descripción funcional

Pantalla de acceso al sistema. El usuario ingresa su email y contraseña; el servlet valida las credenciales contra las tablas USUARIO y PERSONA y, si son correctas y el usuario está activo, crea la sesión (userDni, userNombre, userRol) y redirige al panel correspondiente a su rol.

Botones y redirecciones

- **Entrar** → POST /LoginServlet. Según rol: ADMIN → /admin/home.jsp, DOCENTE → /docente/home.jsp, ALUMNO → /alumno/home.jsp

Reglas de negocio

- La autenticación se realiza por email + contraseña (SELECT sobre USUARIO JOIN PERSONA).
- El rol se normaliza a mayúsculas y determina la redirección.
- La contraseña inicial de toda persona creada por el administrador es igual a su DNI.
- Los campos Email y Contraseña son obligatorios (required) y se les aplica trim().

Validaciones y mensajes de error exactos

- Credenciales no encontradas: *“Usuario o contraseña incorrecto”*
- Usuario con activo = false: *“Usuario inactivo.”*

- Error de conexión a la base de datos: *“No es posible acceder a la base de datos”*

Mensajes de éxito (OK)

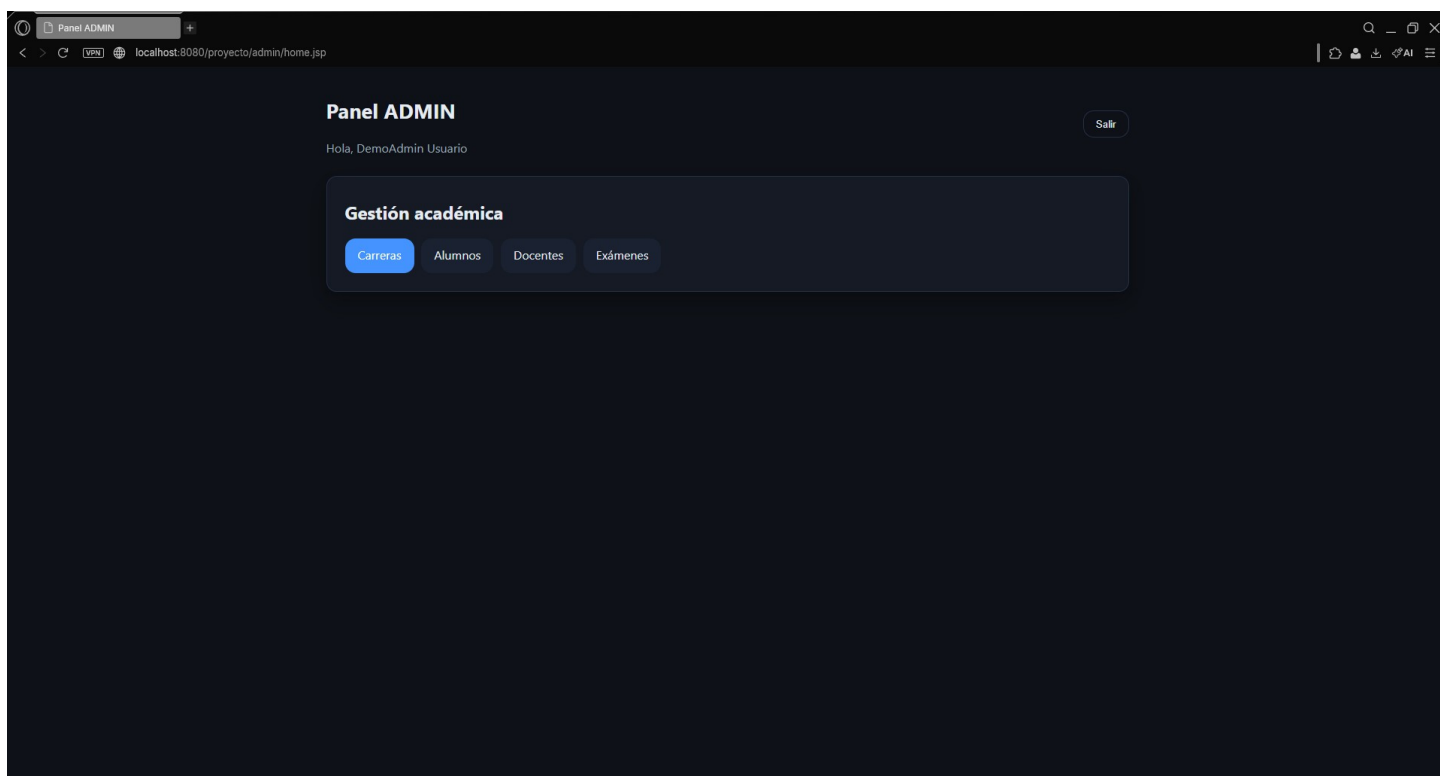
- *En autenticación correcta no muestra banner: redirige directamente al panel del rol (ADMIN, DOCENTE o ALUMNO).*

Usuarios demo (tablas persona / usuario del script SQL)

DNI	Email	Contraseña	tipoUsuario
0	admin@demo	admin	ADMIN
1	docente@demo	docente	DOCENTE
2	alumno@demo	alumno	ALUMNO

Los usuarios demo figuran en el script SQL: DemoAdmin Usuario (DNI 0), demo docente (DNI 1) y Demo Alumno (DNI 2). Los alumnos adicionales creados durante la demostración (alumno2 DNI 3, alumno3 DNI 4) reciben como contraseña su propio DNI.

2. Home Admin



Servlet / JSP: admin/home.jsp

Descripción funcional

Panel principal del administrador. Muestra el saludo “Hola, <nombre>” (en la captura, DemoAdmin Usuario) y el menú de gestión académica. La página verifica la sesión: si el rol no es ADMIN redirige a login.jsp.

Botones y redirecciones

- **Carreras** → /carrera/listar
- **Alumnos** → /alumno/listar
- **Docentes** → /docente/listar
- **Exámenes** → /examen/listar

- **Salir** → POST a /login.jsp

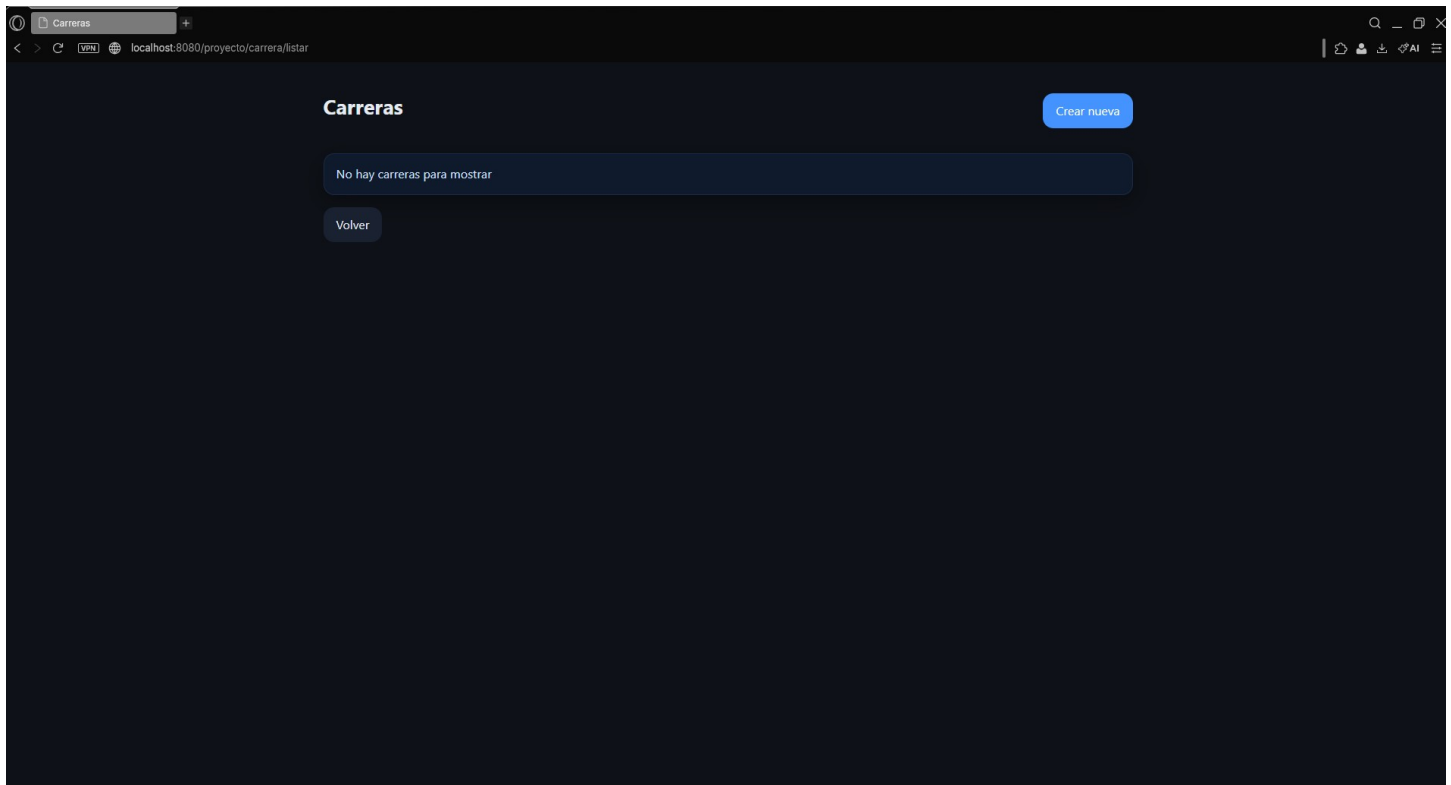
Reglas de negocio

- Guard de seguridad: sin sesión o rol $\neq$ ADMIN → redirección a login.jsp.
- Puede mostrar un mensaje de error recibido por parámetro (error).

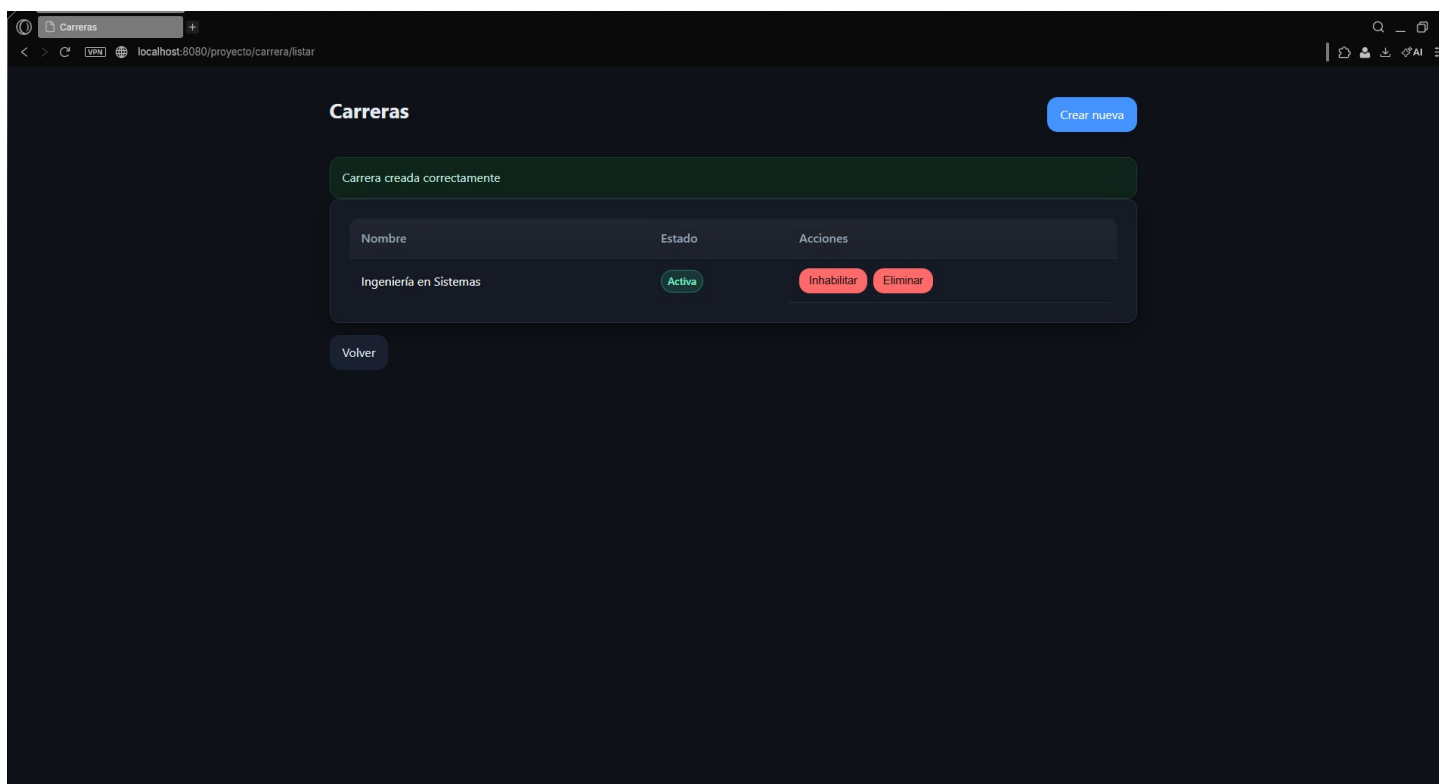
Mensajes de éxito (OK)

- *No muestra mensajes de éxito propios.*

3. Listar carreras



Estado inicial sin carreras cargadas.



Listado con una carrera creada y mensaje de éxito.

Servlet / JSP: AdminListarCarrera (GET) – rol ADMIN

Descripción funcional

Lista todas las carreras del sistema. Cada nombre es un enlace al listado de cursos de esa carrera. Permite habilitar/inhabilitar (baja lógica) y eliminar carreras.

Columnas de la tabla: Nombre (enlace a /curso/listar?idCarrera=), Estado (Activa / Inhabilitada), Acciones

Botones y redirecciones

- **Crear nueva** → /carrera/crear
- **Inhabilitar (por fila)** → POST /objeto/habilitar (tipo=carrera, action=inhabilitar) – confirma con popup
- **Habilitar (por fila)** → POST /objeto/habilitar (tipo=carrera, action=habilitar) – confirma con popup
- **Eliminar (por fila)** → POST /objeto/eliminar (tipo=carrera) – confirma con popup
- **Nombre de carrera** → /curso/listar?idCarrera=
- **Volver** → /admin/home.jsp

Cambios de estado que genera la acción

- Habilitar / Inhabilitar cambia el campo Activo de la carrera (1/0); es una baja lógica, no elimina datos.

Reglas de negocio

- Solo se puede eliminar una carrera si no tiene cursos asociados.
- El estado se muestra con badge: Activa (verde) o Inhabilitada (gris).

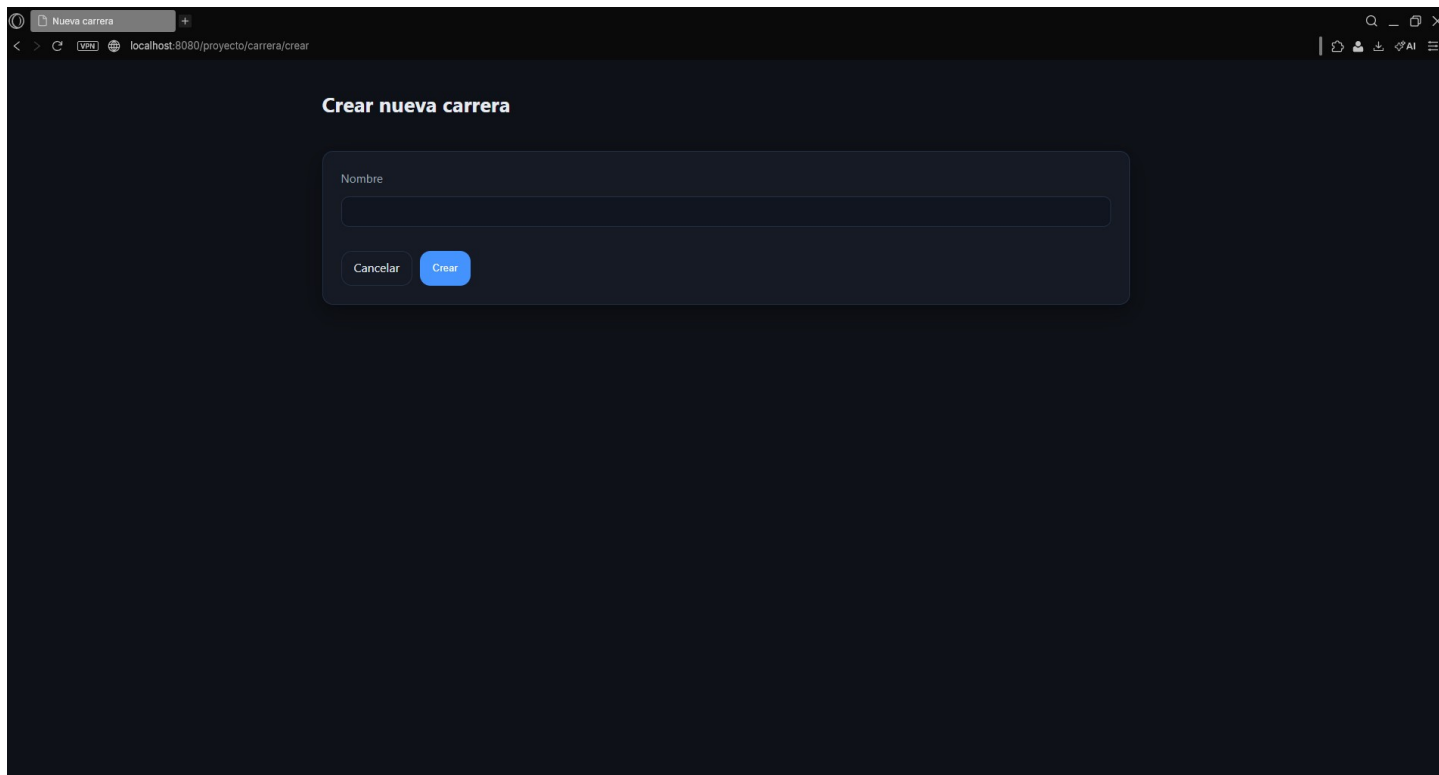
Validaciones y mensajes de error exactos

- Eliminar carrera con cursos: *“No se puede eliminar porque hay cursos asociados”*
- Sin registros: *“No hay carreras para mostrar”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- «Carrera creada correctamente» (al volver de crear una carrera).
- «carrera habilitado/a correctamente» / «carrera inhabilitado/a correctamente».
- «carrera eliminado/a correctamente».

4. Crear carrera



Servlet / JSP: AdminCrearCarrera (GET/POST) – rol ADMIN

Descripción funcional

Formulario de alta de una nueva carrera. Valida que el nombre no esté vacío ni duplicado antes de insertarla (Activo = 1).

Botones y redirecciones

- **Crear** → POST /carrera/crear → éxito: /carrera/listar?ok=Carrera+creada+correctamente
- **Cancelar** → /carrera/listar

Reglas de negocio

- El campo Nombre es obligatorio (required, maxlength 100).
- No se permiten nombres de carrera duplicados.
- La carrera se crea siempre activa (Activo = 1).

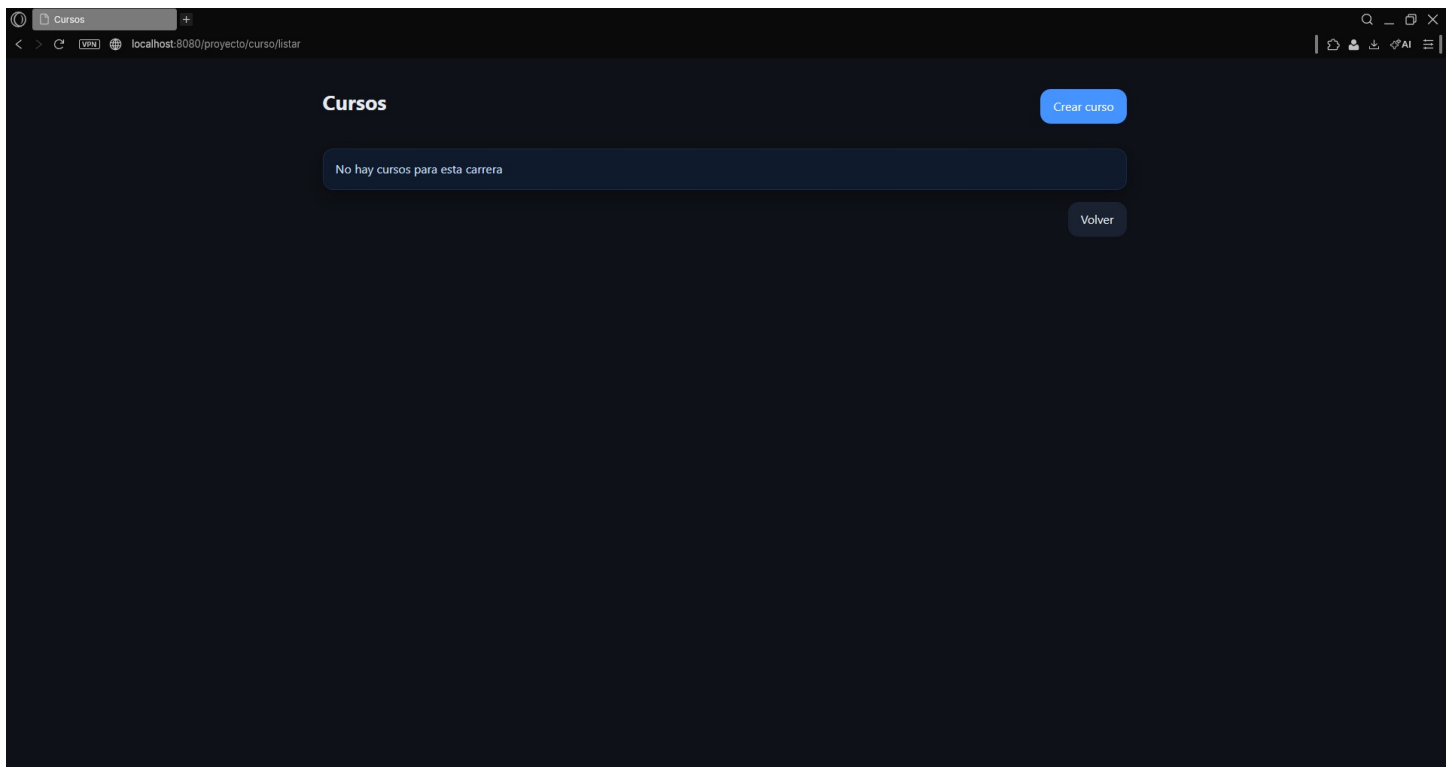
Validaciones y mensajes de error exactos

- Nombre vacío: *“El nombre es obligatorio.”*
- Nombre duplicado: *“Ya existe una carrera con ese nombre.”*
- Excepción al insertar: *“Error al crear la carrera”*

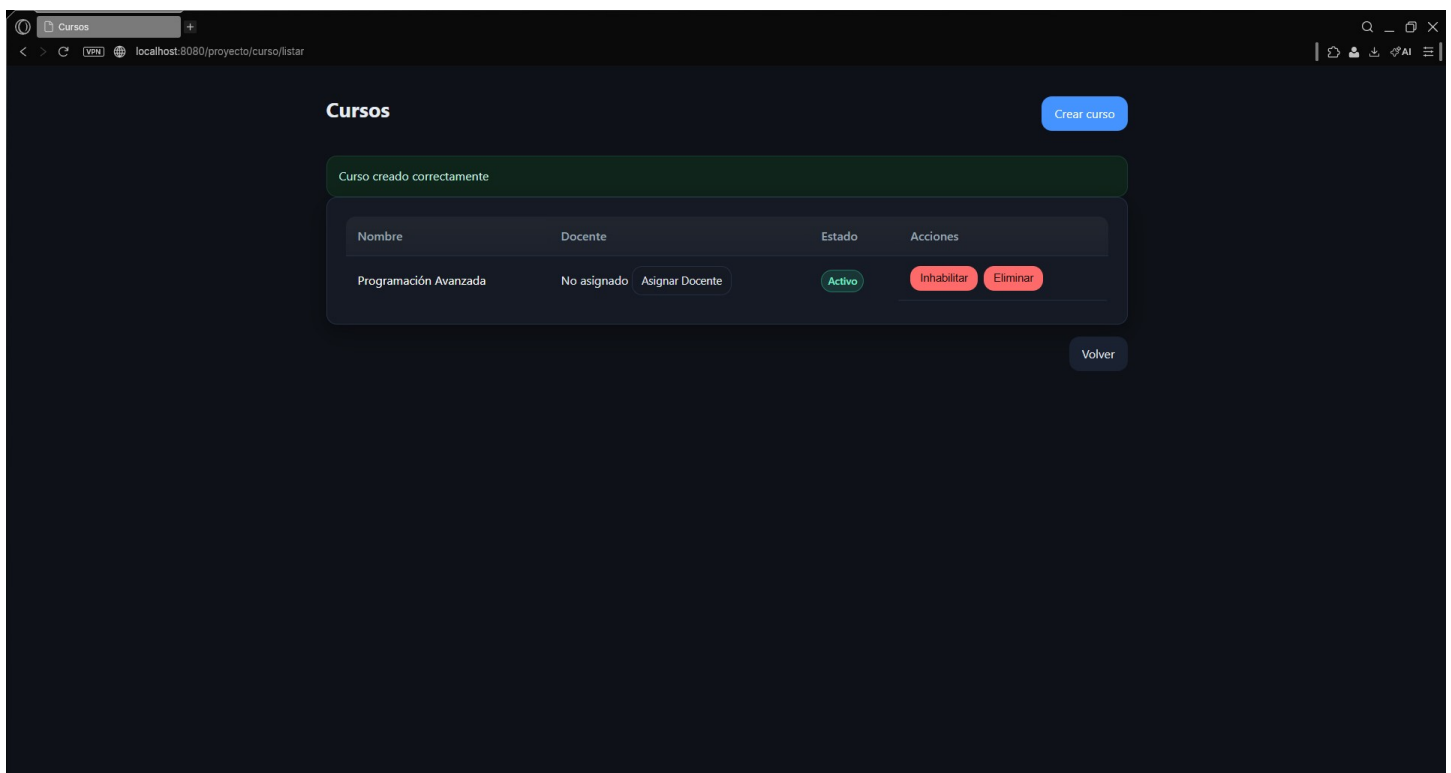
Mensajes de éxito (OK)

- *Al crear correctamente redirige a Listar carreras con «Carrera creada correctamente».*

5. Listar cursos



Carrera sin cursos.



Curso creado, sin docente asignado.

Servlet / JSP: AdminListarCurso (GET) – rol ADMIN**Descripción funcional**

Lista los cursos de una carrera, mostrando el docente asignado (o “No asignado”). Cada nombre enlaza a las comisiones del curso. Permite habilitar/inhabilitar y eliminar cursos y asignar/modificar el docente.

Columnas de la tabla: Nombre (enlace a /comision/listar?idCurso=), Docente (+ botón Asignar/Modificar), Estado, Acciones

Botones y redirecciones

- **Crear curso** → /curso/crear?idCarrera=
- **Asignar Docente / Modificar Docente** → /curso/asignarDocente?idCurso=
- **Inhabilitar / Habilitar (por fila)** → POST /objeto/habilitar (tipo=curso, idPadre=idCarrera) – con popup
- **Eliminar (por fila)** → POST /objeto/eliminar (tipo=curso, idPadre=idCarrera) – con popup
- **Volver** → /carrera/listar

Cambios de estado que genera la acción

- Habilitar/Inhabilitar cambia el campo Activo del curso (baja lógica).

Reglas de negocio

- El botón muestra “Asignar Docente” si dniDocente es nulo y “Modificar Docente” si ya tiene uno.
- No se puede eliminar un curso con docente asignado ni con alumnos inscriptos en sus comisiones.

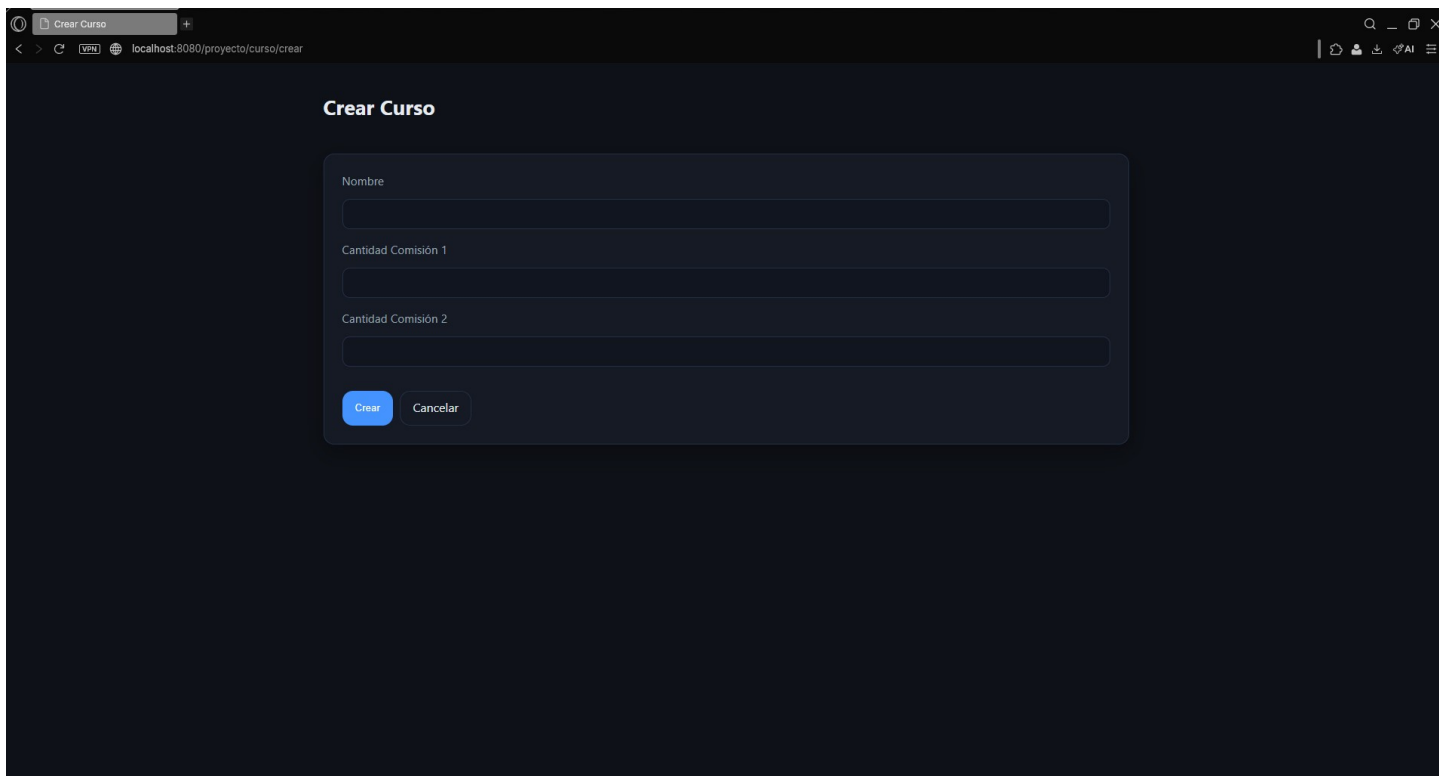
Validaciones y mensajes de error exactos

- Eliminar curso con docente: *“No se puede eliminar porque hay un docente asignado”*
- Eliminar curso con alumnos: *“No se puede eliminar porque hay alumnos inscriptos”*
- Carrera sin cursos: *“No hay cursos para esta carrera”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *«Curso creado correctamente» (al volver de crear un curso).*
- *«Docente modificado correctamente» (al asignar o quitar docente).*
- *«curso habilitado/a correctamente» / «curso inhabilitado/a correctamente».*
- *«curso eliminado/a correctamente».*

6. Crear curso



Servlet / JSP: AdminCrearCurso (GET/POST) – rol ADMIN

Descripción funcional

Formulario de alta de curso. Además del curso, crea automáticamente sus DOS comisiones con los cupos indicados. Verifica que no exista otro curso con el mismo nombre en la misma carrera.

Botones y redirecciones

- **Crear** → POST /curso/crear → éxito: /curso/listar?idCarrera=...&ok=Curso+creado+correctamente
- **Cancelar** → /curso/listar?idCarrera=

Cambios de estado que genera la acción

- El alta inserta 1 curso (Activo=1) y 2 comisiones (Activo=1) con los cupos Comisión 1 y Comisión 2.

Reglas de negocio

- Nombre obligatorio; Cantidad Comisión 1 y 2 deben ser enteros positivos (mayores a 0).
- El nombre de curso debe ser único dentro de la carrera.

Validaciones y mensajes de error exactos

- Campos faltantes: *“Todos los campos son obligatorios.”*
- Cupos no numéricos: *“Los campos numéricos son inválidos.”*
- Cupo ≤ 0 : *“El cupo de cada comisión debe ser mayor a 0.”*
- Curso duplicado en la carrera: *“Ya existe un curso con ese nombre en la carrera seleccionada.”*
- Excepción al insertar: *“Error al crear el curso”*

Mensajes de éxito (OK)

- *Al crear correctamente redirige a Listar cursos con «Curso creado correctamente».*

7. Asignar docente

Asignar Docente

Curso: Programación Avanzada

Seleccionar	Docente	DNI
☐	docente, demo	1

Asignar docente Volver

Servlet / JSP: AdminAsignarDocente (GET/POST) – rol ADMIN

Descripción funcional

Permite asignar, modificar o quitar el docente de un curso. Muestra el curso y la lista de docentes activos con un radio por fila; el docente ya asignado aparece marcado.

Columnas de la tabla: Seleccionar (radio), Docente (Apellido, Nombre), DNI

Botones y redirecciones

- **Asignar docente** → POST /curso/asignarDocente (accion=asignar) → /curso/listar?idCarrera=...&ok=Docente+modificado+correctamente
- **Quitar docente (solo si hay uno asignado)** → POST /curso/asignarDocente (accion=quitar) → UPDATE dniDocente = NULL
- **Volver** → /curso/listar?idCarrera=

Cambios de estado que genera la acción

- Asignar: UPDATE curso SET dniDocente = <dni>. Quitar: UPDATE curso SET dniDocente = NULL.

Reglas de negocio

- Solo se listan docentes con usuario activo.
- El radio del docente actualmente asignado se muestra marcado (checked).

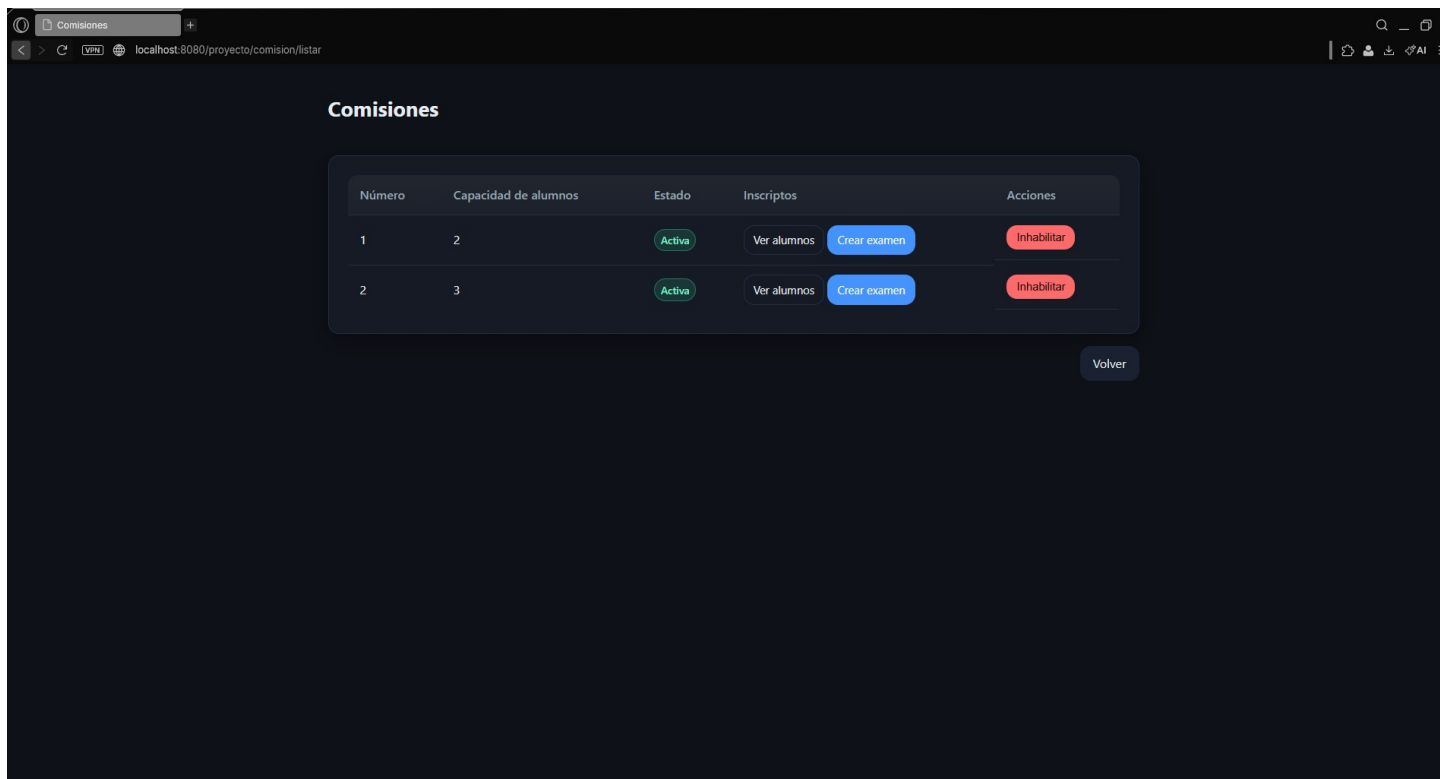
Validaciones y mensajes de error exactos

- Asignar sin seleccionar docente: *“Debe seleccionar un docente”*
- Error en el UPDATE: *“Error al actualizar docente”*
- Error al cargar la lista: *“Error al cargar docentes.”*
- Sin docentes activos: *“No hay docentes disponibles.”*

Mensajes de éxito (OK)

- *Al asignar o quitar el docente redirige a Listar cursos con «Docente modificado correctamente».*

8. Listar comisiones



Servlet / JSP: AdminListarComision (GET) – rol ADMIN

Descripción funcional

Lista las comisiones de un curso con su número, cupo y estado. Desde aquí se accede a los alumnos de la comisión y a la creación de exámenes.

Columnas de la tabla: Número, Capacidad de alumnos, Estado, Inscriptos (Ver alumnos / Crear examen), Acciones

Botones y redirecciones

- **Ver alumnos** → /comision/alumnos?idComision=
- **Crear examen** → /examen/crear?idComision=...&idCurso=
- **Inhabilitar / Habilitar (por fila)** → POST /objeto/habilitar (tipo=comision, idPadre=idCurso) – con popup
- **Volver** → /curso/listar?idCarrera=

Cambios de estado que genera la acción

- Habilitar/Inhabilitar cambia el campo Activo de la comisión (baja lógica).

Reglas de negocio

- El número de comisión se calcula con ROW_NUMBER() particionado por curso (no se almacena en la tabla).
- Las comisiones no tienen acción “Eliminar” propia: se eliminan en cascada al eliminar el curso.

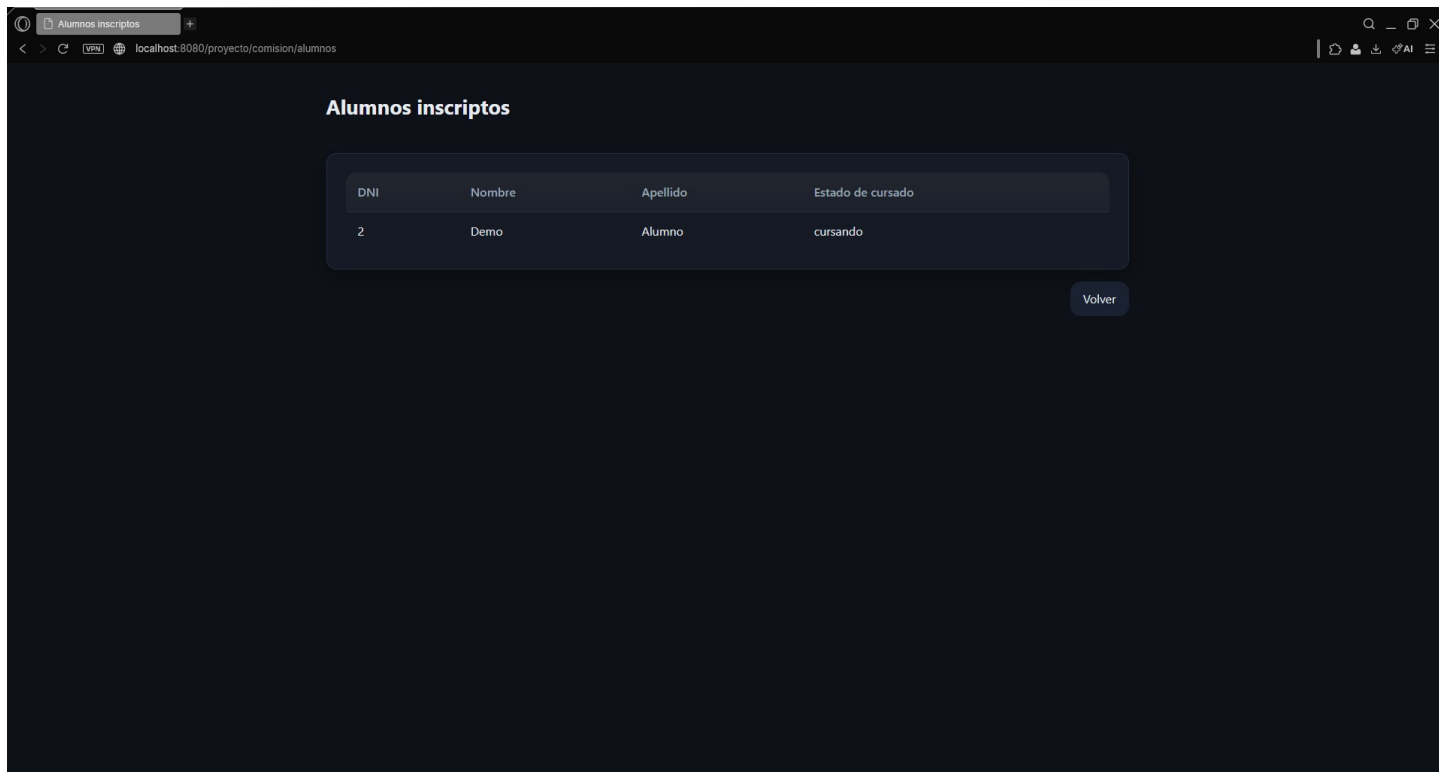
Validaciones y mensajes de error exactos

- Curso sin comisiones: *“No hay comisiones para este curso”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- «comision habilitado/a correctamente» / «comision inhabilitado/a correctamente».

9. Alumnos de comisión



Servlet / JSP: AdminListarAlumnoComision (GET) – rol ADMIN

Descripción funcional

Vista de solo lectura con los alumnos inscriptos en una comisión y su estado de cursado (cursando, regular, libre, promocionado, aprobado).

Columnas de la tabla: DNI, Nombre, Apellido, Estado de cursado

Botones y redirecciones

- **Volver** → /comision/listar?idCurso=

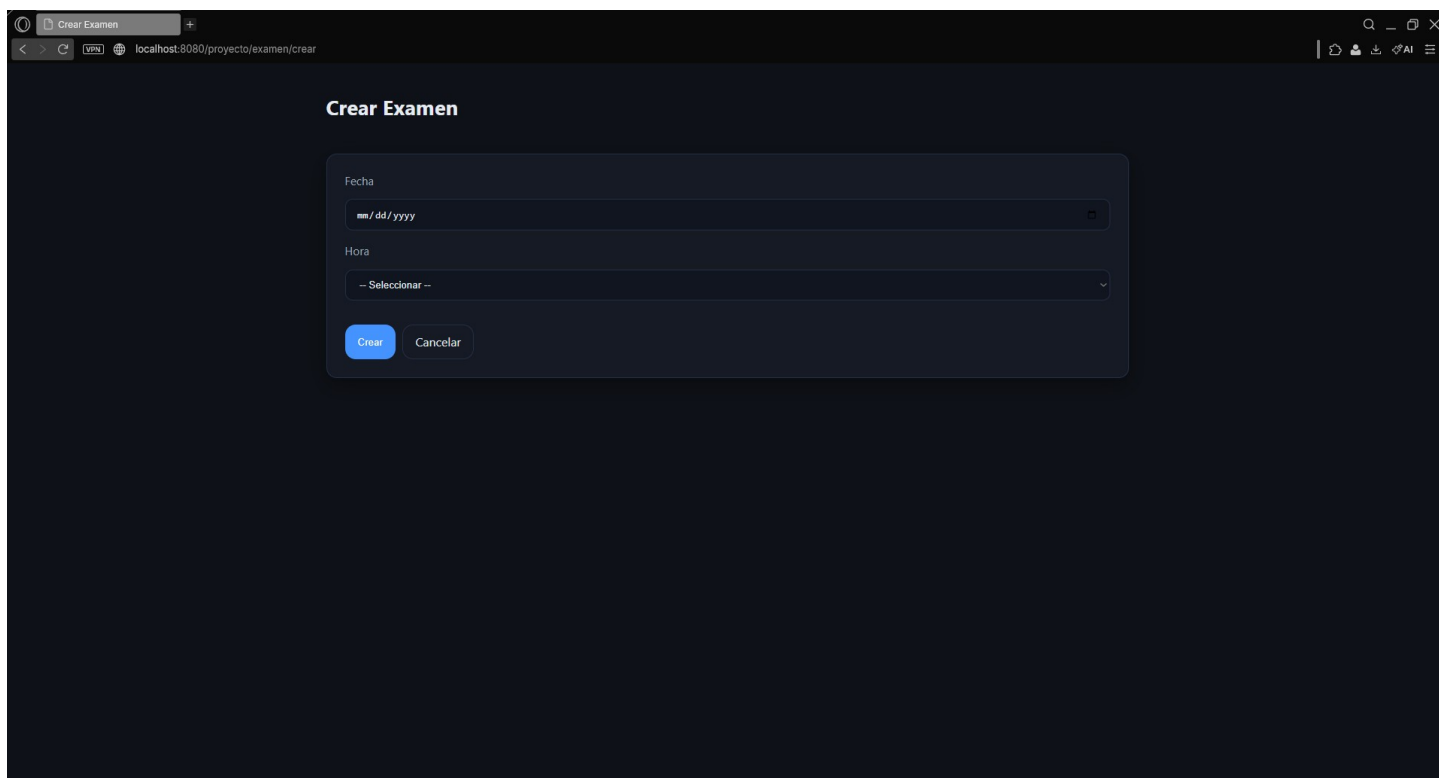
Validaciones y mensajes de error exactos

- Comisión sin alumnos: *“No hay alumnos inscriptos en esta comisión.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *No muestra mensajes de éxito (vista de solo lectura).*

10. Crear examen



Servlet / JSP: AdminCrearExamen (GET/POST) – rol ADMIN

Descripción funcional

Formulario de alta de examen para una comisión. La fecha mínima es hoy + 2 días y la hora se elige de una lista de 07:00 a 21:00 cada 30 minutos. El docente del examen se toma automáticamente del docente del curso.

Botones y redirecciones

- **Crear** → POST /examen/crear → éxito: /examen/listar?ok=Exámen+creado+correctamente
- **Cancelar** → /comision/listar?idCurso=

Cambios de estado que genera la acción

- Inserta el examen con estado inicial “pendiente” y el dniDocente heredado del curso.

Reglas de negocio

- La fecha del examen debe ser a partir de hoy + 2 días (no se permite para hoy ni mañana).
- No puede existir otro examen de la misma comisión en la misma fecha.
- El curso de la comisión debe tener docente asignado.

Validaciones y mensajes de error exactos

- idComisión inválido: *“Datos invalidos”*
- Fecha u hora faltante: *“Fecha y hora son requeridos”*
- Formato de fecha inválido: *“Formato de fecha inválido (YYYY-MM-DD)”*
- Formato de hora inválido: *“Formato de hora inválido (HH:MM)”*
- Fecha anterior al mínimo: *“Fecha debe ser a partir del dd/MM/yyyy”*
- Examen duplicado en la fecha: *“Ya existe un exámen para esa fecha”*
- Comisión inexistente: *“No se encontró la comisión”*

- Curso sin docente: *“El curso no tiene docente asignado. Asigná un docente antes de crear el examen”*
- Excepción al insertar: *“Error al crear el exámen”*

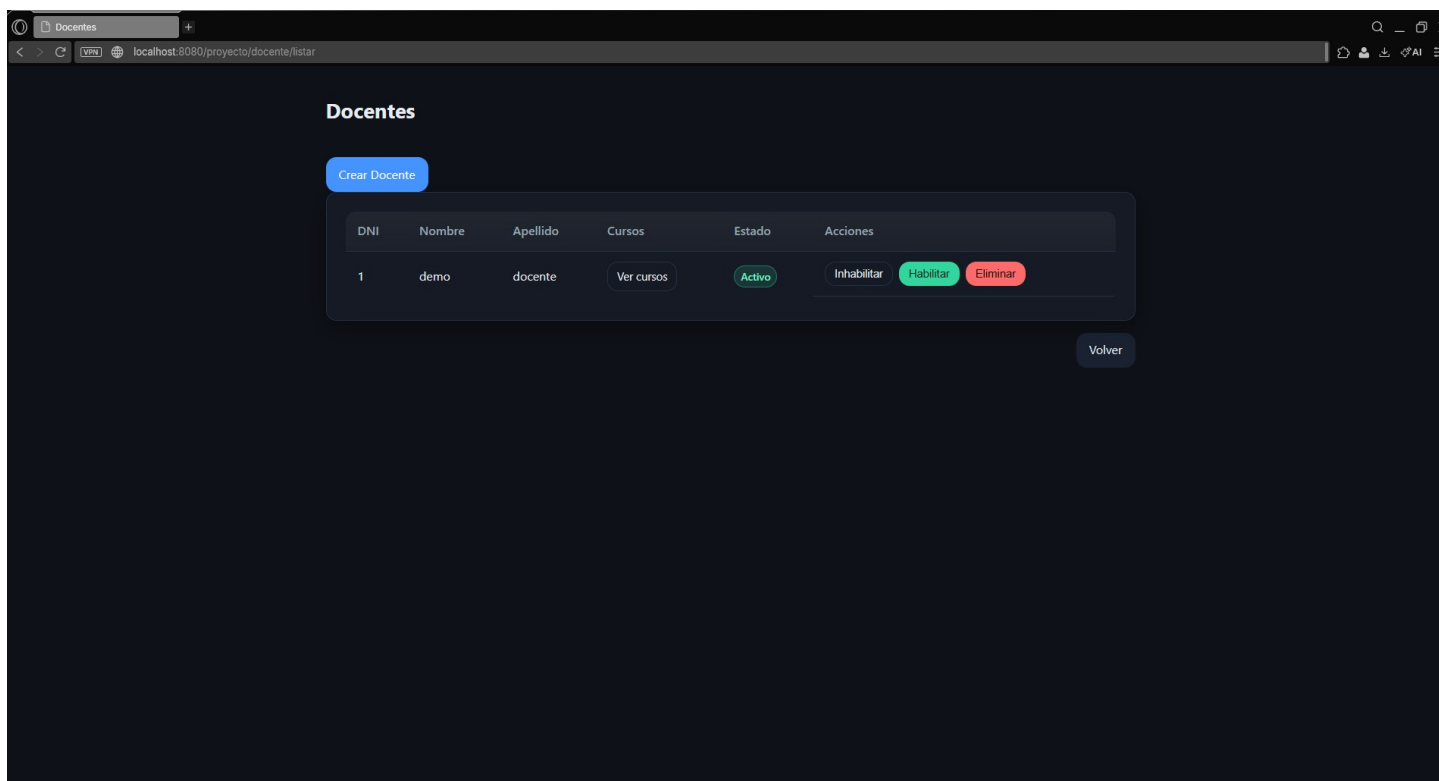
Mensajes de éxito (OK)

- *Al crear correctamente redirige a Listar exámenes con «Exámen creado correctamente».*

Notas

- Formato de fecha: el campo Fecha trabaja con formato DD/MM/AAAA. En la captura se ve como mm/dd/yyyy porque el navegador está configurado en inglés y fuerza esa visualización del control de fecha (no es modificable desde la aplicación). De todas formas el sistema interpreta y guarda la fecha correctamente en ambos casos: internamente el input date entrega el valor en formato YYYY-MM-DD, que es el que valida y persiste el servlet.

11. Listar docentes



Servlet / JSP: AdminListarDocente (GET) – rol ADMIN

Descripción funcional

Lista todos los docentes con su estado (activo/inhabilitado) y acceso a sus cursos. Permite crear, habilitar/inhabilitar y eliminar docentes.

Columnas de la tabla: DNI, Nombre, Apellido, Cursos (Ver cursos), Estado, Acciones

Botones y redirecciones

- **Crear Docente** → /persona/crear?rol=docente
- **Ver cursos** → /admin/docente/cursos?dni=
- **Inhabilitar / Habilitar (por fila)** → POST /persona/habilitar (rol=docente) – con popup
- **Eliminar (por fila)** → POST /persona/eliminar (rol=docente) – con popup
- **Volver** → /admin/home.jsp

Cambios de estado que genera la acción

- Habilitar/Inhabilitar cambia el campo activo del USUARIO (1/0).

Reglas de negocio

- El botón Inhabilitar se deshabilita si el docente ya está inactivo; Habilitar se deshabilita si ya está activo.
- No se puede eliminar un docente que tenga cursos asociados.

Validaciones y mensajes de error exactos

- Eliminar docente con cursos: *“No se puede eliminar porque tiene cursos asociados”*
- Sin docentes: *“No hay docentes para mostrar”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *«docente creado correctamente» (al volver de crear un docente).*
- *«docente habilitado correctamente» / «docente inhabilitado correctamente».*
- *«docente eliminado correctamente».*

12. Crear docente

Servlet / JSP: AdminCrearPersona (GET/POST, rol=docente) – rol ADMIN

Descripción funcional

Formulario de alta de docente. Crea el registro en PERSONA y su cuenta en USUARIO con tipoUsuario DOCENTE, activo=1 y contraseña inicial igual al DNI.

Columnas de la tabla: Campos: DNI*, Nombre*, Apellido*, Dirección, Email*, Fecha de Nacimiento, Sexo, Teléfono

Botones y redirecciones

- **Crear** → POST /persona/crear → éxito: /docente/listar?ok=docente+creado+correctamente
- **Cancelar** → /docente/listar

Cambios de estado que genera la acción

- Inserta una fila en PERSONA y una en USUARIO (tipoUsuario=DOCENTE, activo=1, password=DNI).

Reglas de negocio

- DNI, Nombre, Apellido y Email son obligatorios; el DNI debe ser numérico (1 a 99.999.999).
- No se permiten DNI ni email duplicados.
- El sexo se normaliza a M / F / O.

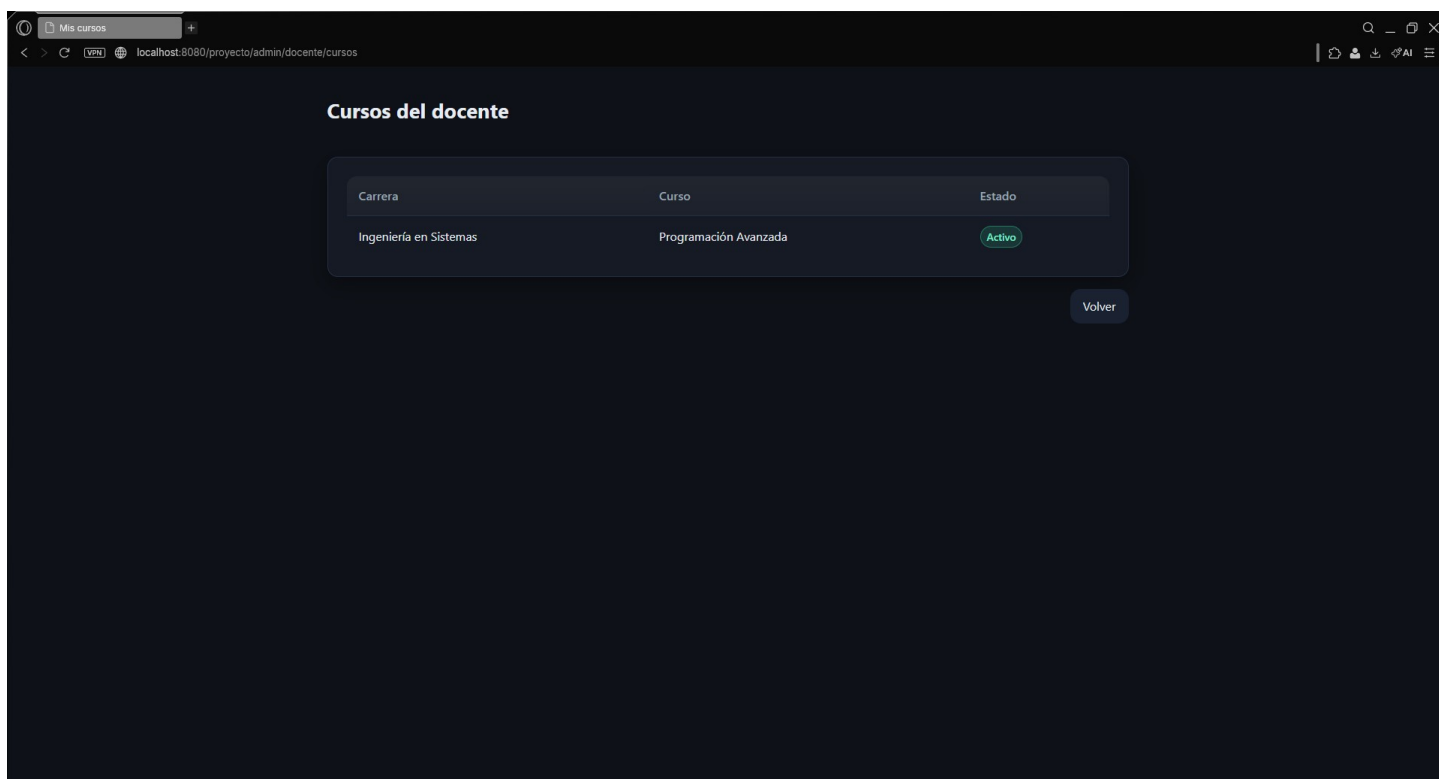
Validaciones y mensajes de error exactos

- Campos obligatorios vacíos: *“Debe ingresar dni, nombre, apellido y email.”*
- DNI no numérico: *“El DNI debe ser un número válido.”*
- DNI duplicado: *“Ya existe una persona con ese DNI.”*
- Email duplicado: *“Ya existe una persona con ese email.”*
- Excepción al insertar: *“Error al crear persona.”*

Mensajes de éxito (OK)

- *Al crear correctamente redirige a Listar docentes con «docente creado correctamente».*

13. Cursos del docente



Servlet / JSP: AdminListarCursoDocente (GET) – rol ADMIN

Descripción funcional

Vista (desde la perspectiva del administrador) de los cursos asignados a un docente, con la carrera a la que pertenecen y su estado.

Columnas de la tabla: Carrera, Curso, Estado

Botones y redirecciones

- **Volver** → /docente/listar

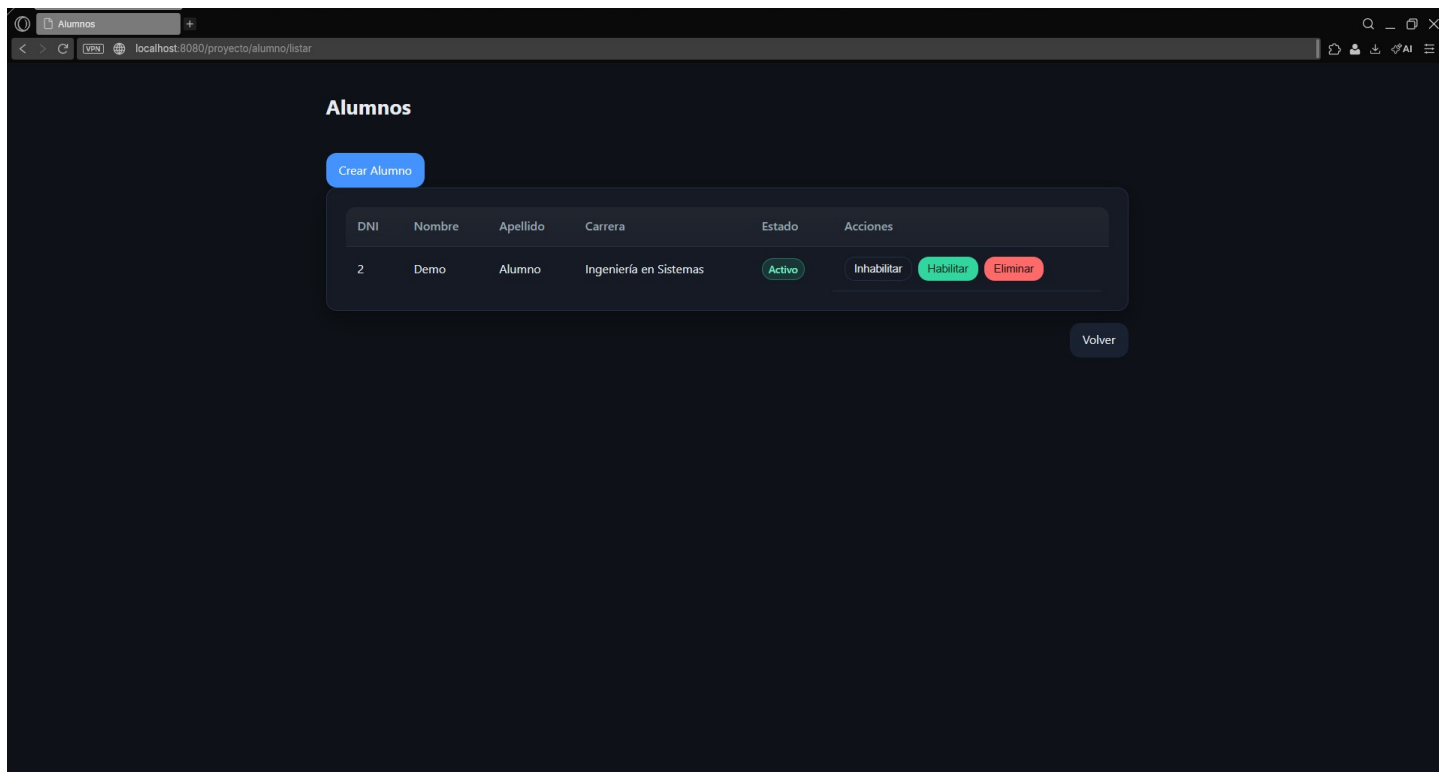
Validaciones y mensajes de error exactos

- Docente sin cursos: *“El docente no tiene cursos asignados.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *No muestra mensajes de éxito.*

14. Listar alumnos



Servlet / JSP: AdminListarAlumno (GET) – rol ADMIN

Descripción funcional

Lista todos los alumnos con su carrera inscrita (o “Sin Carrera”) y su estado. Permite crear, habilitar/inhabilitar y eliminar alumnos.

Columnas de la tabla: DNI, Nombre, Apellido, Carrera, Estado, Acciones

Botones y redirecciones

- **Crear Alumno** → /persona/crear?rol=alumno
- **Inhabilitar / Habilitar (por fila)** → POST /persona/habilitar (rol=alumno) – con popup
- **Eliminar (por fila)** → POST /persona/eliminar (rol=alumno) – con popup
- **Volver** → /admin/home.jsp

Cambios de estado que genera la acción

- Habilitar/Inhabilitar cambia el campo activo del USUARIO.

Reglas de negocio

- Se muestra la carrera con estado INSCRIPTO; si no tiene, “Sin Carrera”.
- No se puede eliminar un alumno asociado a una carrera.

Validaciones y mensajes de error exactos

- Eliminar alumno con carrera: *“No se puede eliminar porque está asociado a una carrera”*
- Sin alumnos: *“No hay alumnos para mostrar”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *«alumno creado correctamente» (al volver de crear un alumno).*
- *«alumno habilitado correctamente» / «alumno inhabilitado correctamente».*
- *«alumno eliminado correctamente».*

15. Crear alumno

Servlet / JSP: AdminCrearPersona (GET/POST, rol=alumno) – rol ADMIN

Descripción funcional

Formulario de alta de alumno. Crea PERSONA y USUARIO con tipoUsuario ALUMNO, activo=1 y contraseña inicial igual al DNI. En el alta de alumno la Fecha de Nacimiento es obligatoria.

Columnas de la tabla: Campos: DNI*, Nombre*, Apellido*, Dirección, Email*, Fecha de Nacimiento*, Sexo, Teléfono

Botones y redirecciones

- **Crear** → POST /persona/crear → éxito: /alumno/listar?ok=alumno+creado+correctamente
- **Cancelar** → /alumno/listar

Cambios de estado que genera la acción

- Inserta una fila en PERSONA y una en USUARIO (tipoUsuario=ALUMNO, activo=1, password=DNI).

Reglas de negocio

- Mismas validaciones de unicidad (DNI y email) que el alta de docente.

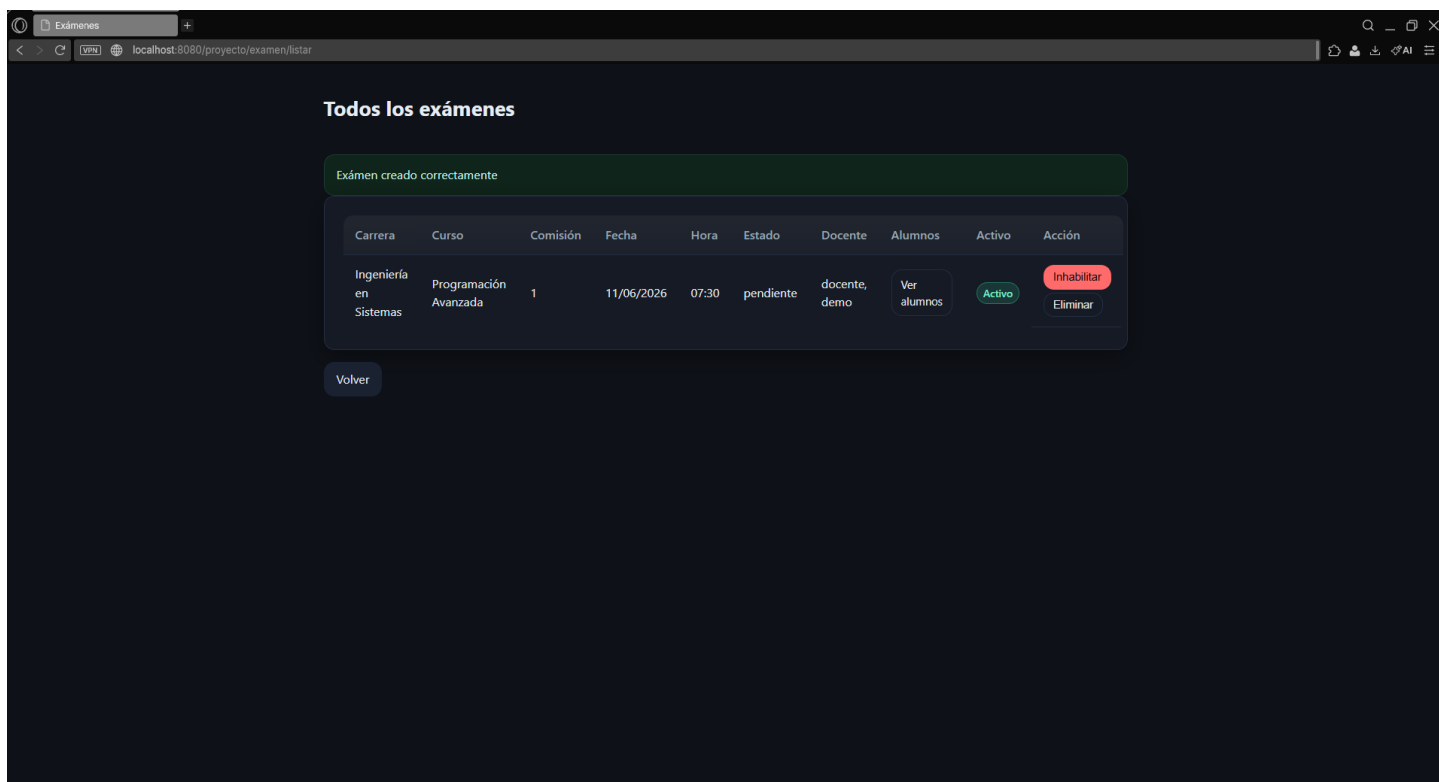
Validaciones y mensajes de error exactos

- Campos obligatorios vacíos: *“Debe ingresar dni, nombre, apellido y email.”*
- DNI no numérico: *“El DNI debe ser un número válido.”*
- DNI duplicado: *“Ya existe una persona con ese DNI.”*
- Email duplicado: *“Ya existe una persona con ese email.”*
- Excepción al insertar: *“Error al crear persona.”*

Mensajes de éxito (OK)

- *Al crear correctamente redirige a Listar alumnos con «alumno creado correctamente».*

16. Listar exámenes (admin) – estado pendiente



Servlet / JSP: AdminListarExamen (GET) – rol ADMIN

Descripción funcional

Lista todos los exámenes del sistema con carrera, curso, comisión, fecha, hora, estado y docente. En esta captura el examen recién creado figura con estado “pendiente” (su fecha aún no llegó). Permite habilitar/inhabilitar y eliminar exámenes y ver sus alumnos.

Columnas de la tabla: Carrera, Curso, Comisión, Fecha, Hora, Estado, Docente, Alumnos (Ver alumnos), Activo, Acción

Botones y redirecciones

- **Ver alumnos** → /examen/alumnos?idExamen=
- **Inhabilitar / Habilitar (por fila)** → POST /objeto/habilitar (tipo=examen) – con popup
- **Eliminar (por fila)** → POST /objeto/eliminar (tipo=examen) – con popup
- **Volver** → /admin/home.jsp

Cambios de estado que genera la acción

- Ciclo de vida del examen: pendiente → rendido (al cargar asistencia) → cerrado (al cargar notas). También puede pasar a cancelado por el proceso automático (ver Abstract).

- Habilitar/Inhabilitar cambia el campo Activo del examen.

Reglas de negocio

- Solo se puede eliminar un examen si no tiene alumnos inscriptos y si su estado es “pendiente”.

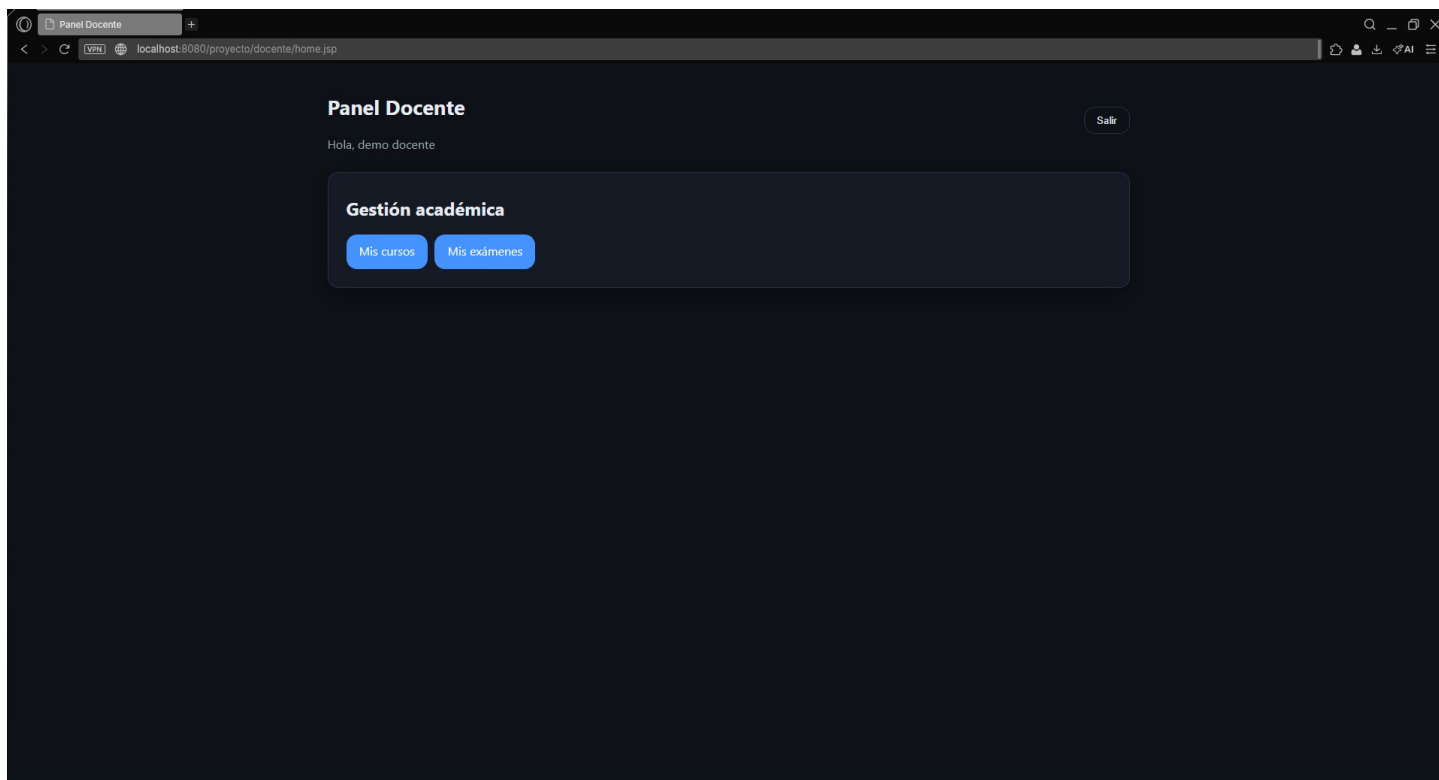
Validaciones y mensajes de error exactos

- Eliminar examen con alumnos: *“No se puede eliminar porque hay alumnos inscriptos”*
- Eliminar examen no pendiente: *“No se puede eliminar porque el examen no esta pendiente”*
- Sin exámenes: *“No hay exámenes registrados.”*
- Error de base de datos: *“Error al acceder a la base de datos.”*

Mensajes de éxito (OK)

- *«Exámen creado correctamente» (al volver de crear un examen).*
- *«examen habilitado/a correctamente» / «examen inhabilitado/a correctamente».*
- *«examen eliminado/a correctamente».*

17. Home Docente



Servlet / JSP: docente/home.jsp

Descripción funcional

Panel principal del docente, “Hola, demo docente”. Guard de seguridad: si el rol no es DOCENTE redirige a login.jsp.

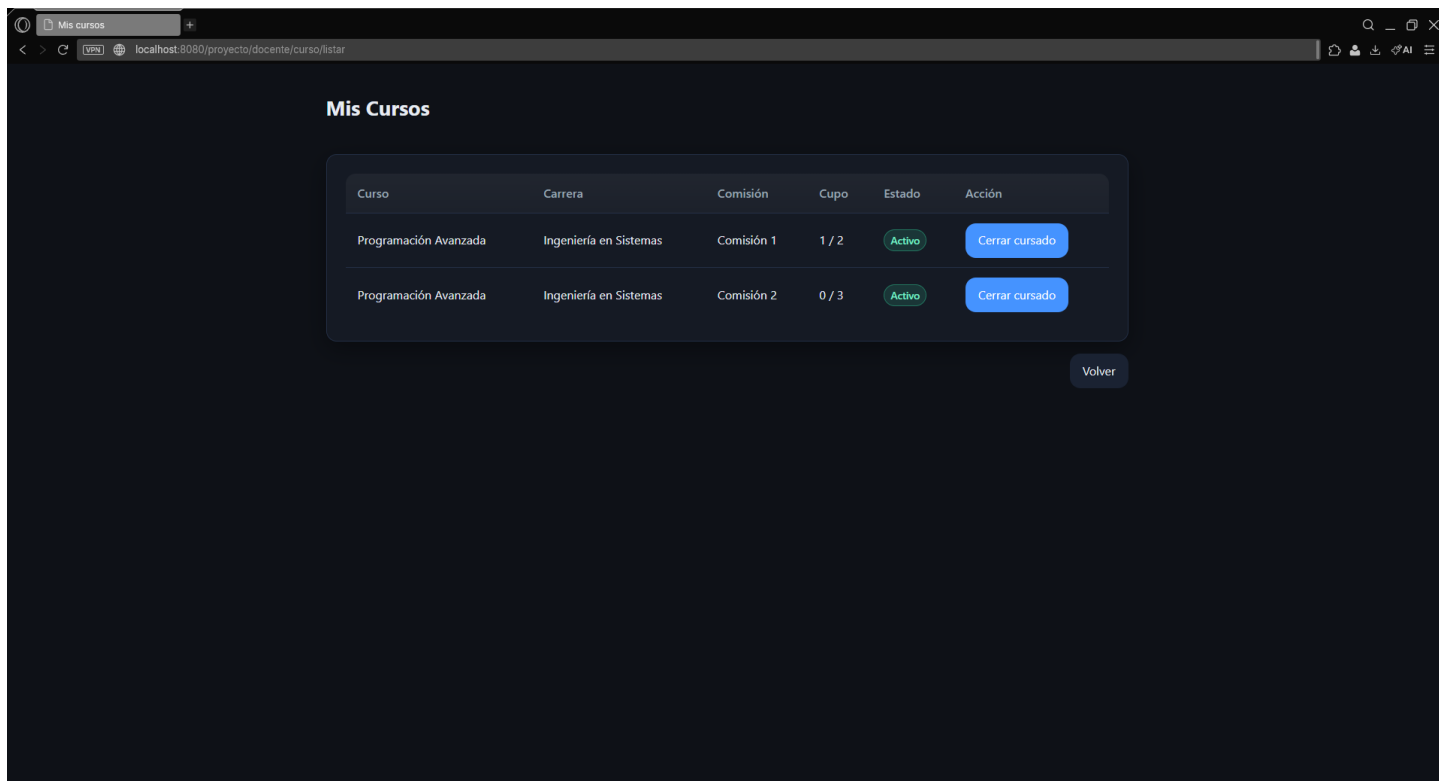
Botones y redirecciones

- **Mis cursos** → /docente/curso/listar
- **Mis exámenes** → /docente/examen/listar
- **Salir** → /login.jsp

Mensajes de éxito (OK)

- *No muestra mensajes de éxito.*

18. Listar cursos (docente)



Servlet / JSP: DocenteListarCurso (GET) – rol DOCENTE

Descripción funcional

Lista los cursos del docente con sus comisiones activas, mostrando el cupo (inscriptos / total) y el acceso al cierre de cursado de cada comisión.

Columnas de la tabla: Curso, Carrera, Comisión, Cupo (inscriptos/cupo), Estado, Acción

Botones y redirecciones

- **Cerrar cursado (si hay comisión)** → /docente/cursos/comision/alumnos?idComision=
- **Volver** → /docente/home.jsp

Reglas de negocio

- Si la comisión no existe/está inactiva, la columna Acción muestra “-”.

Validaciones y mensajes de error exactos

- Sin cursos asignados: *“No tiene cursos asignados.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *«Cursado cerrado correctamente» (al volver del cierre de cursado).*

19. Alumnos de comisión – cierre de cursado

DNI	Alumno	Condición final	Nota (solo promocionado)
2	Alumno, Demo	Libre	
3	demo, alumno2	Regular	
4	demo, alumno3	Promocionado	8

Servlet / JSP: DocenteGestionComision (GET/POST) – rol DOCENTE

Descripción funcional

Pantalla de cierre de cursado de una comisión. El docente asigna a cada alumno una condición final (Libre / Regular / Promocionado) y, solo para los promocionados, una nota. Todo el cierre se ejecuta dentro de una transacción: si algún dato es inválido se hace rollback completo.

Columnas de la tabla: DNI, Alumno, Condición final (Libre/Regular/Promocionado), Nota (solo promocionado)

Botones y redirecciones

- **Cerrar cursado y guardar condiciones** → Abre popup “¿Confirma el cierre del cursado? Esta acción no se puede editar.” → Confirmar: POST → /docente/curso/listar?ok=Cursado+cerrado+correctamente
- **Cancelar (en popup)** → cierra el popup sin guardar
- **Volver** → /docente/curso/listar

Cambios de estado que genera la acción

- comision_alumno.estado pasa de “cursando” a la condición elegida (libre / regular / promocionado); para promocionado se guarda la nota (8–10).
- Si un alumno promocionado completó todos los cursos activos de la carrera, su estado en alumno_carrera pasa a APROBADO.

Reglas de negocio

- El docente solo puede cerrar comisiones de sus propios cursos (validación doble en GET y POST).
- La condición es obligatoria y debe ser uno de los tres valores válidos.
- Solo los promocionados llevan nota; regular y libre quedan con nota nula.
- La nota de promoción debe estar entre 8 y 10.

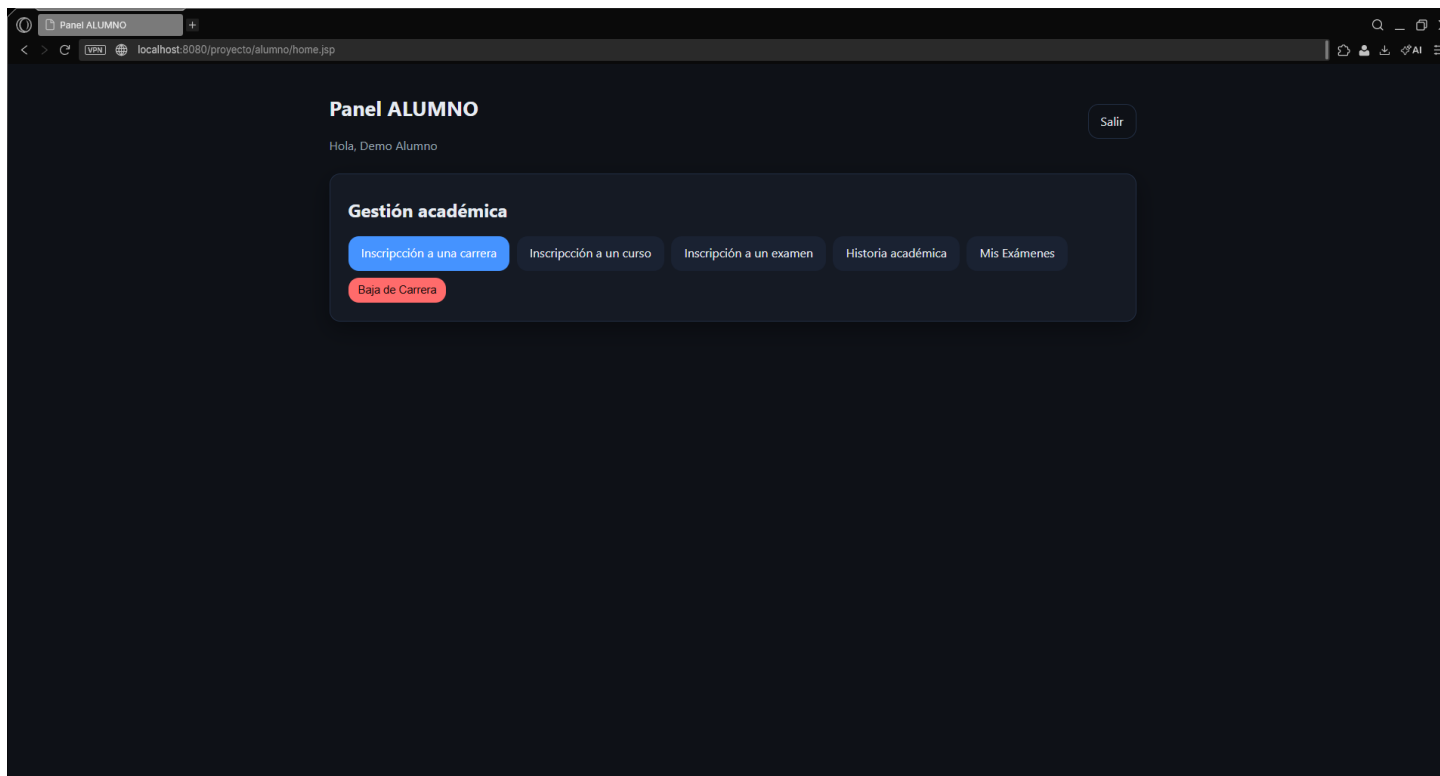
Validaciones y mensajes de error exactos

- Sin permiso (POST): *“No tenés permiso para cerrar esa comisión”*
- Sin permiso (GET): *“No tenés permiso sobre esa comisión”*
- Comisión sin alumnos: *“No hay alumnos para cerrar el cursado”*
- Condición inválida: *“Condición inválida para el alumno con dni X”*
- Nota de promoción fuera de rango: *“El alumno con dni X promocionado debe tener nota entre 8 y 10”*
- Error general: *“No se pudo cerrar el cursado”*
- Comisión inexistente: *“No hay alumnos inscriptos en esta comisión.”*

Mensajes de éxito (OK)

- *Al confirmar redirige a Mis cursos con «Cursado cerrado correctamente».*

20. Home Alumno



Servlet / JSP: alumno/home.jsp

Descripción funcional

Panel principal del alumno, “Hola, <nombre>”. Guard de seguridad: si el rol no es ALUMNO redirige a login.jsp. Concentra las inscripciones (carrera, curso, examen), la historia académica, los exámenes inscriptos y la baja de carrera.

Botones y redirecciones

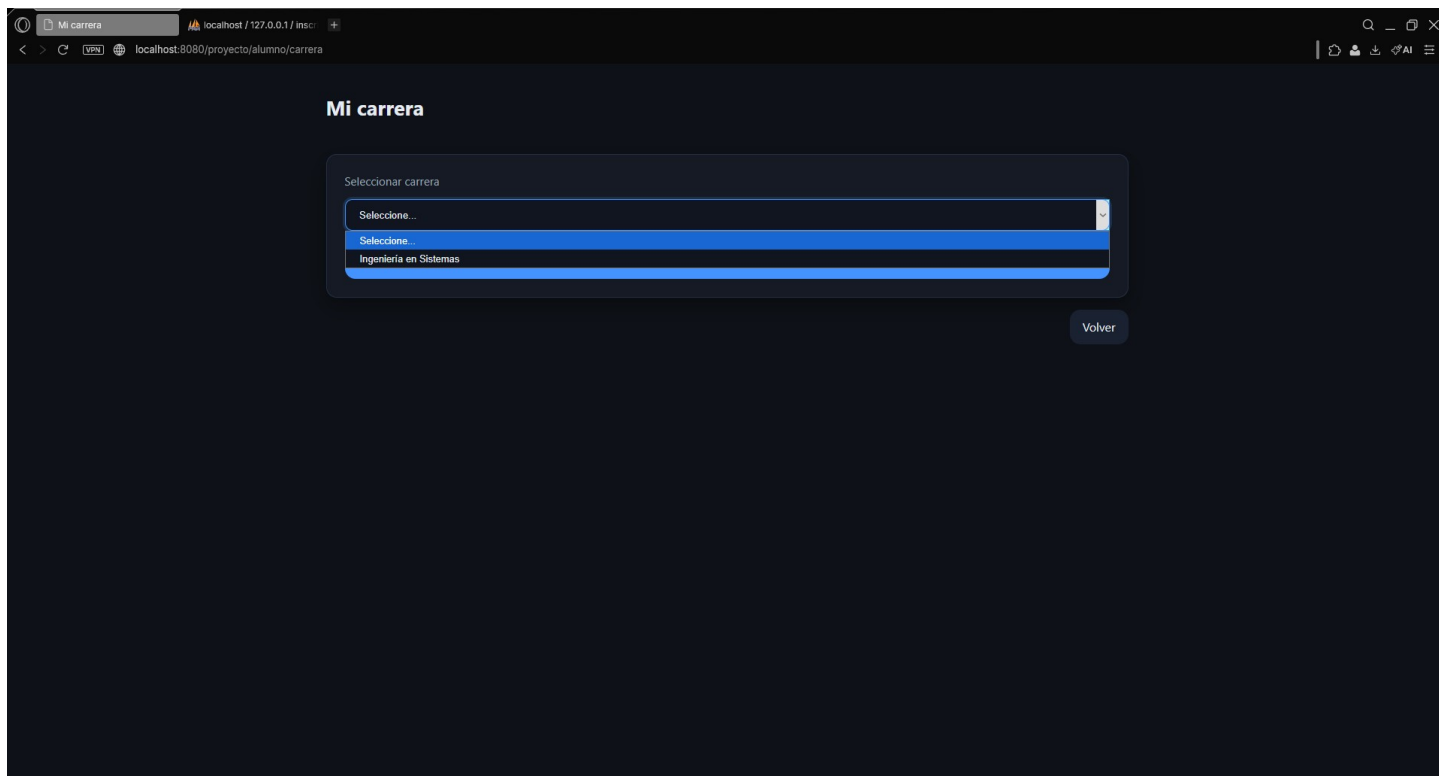
- **Inscripción a una carrera** → /alumno/carrera
- **Inscripción a un curso** → /alumno/cursos
- **Inscripción a un examen** → /alumno/examenes
- **Historia académica** → /alumno/historiaAcademica
- **Mis Exámenes** → /alumno/examenesInscripto
- **Baja de Carrera** → abre el popup de confirmación (ver pantalla 36)

- **Salir** → /login.jsp

Mensajes de éxito (OK)

- «*Inscripción realizada*» (tras inscribirse a una carrera).
- «*Ya estás inscripto a la carrera X*» (si ya tenía una carrera activa).
- «*Baja realizada*» (tras dar de baja la carrera).

21. Inscribir carrera



Servlet / JSP: AlumnoInscribirCarrera (GET/POST) – rol ALUMNO

Descripción funcional

Permite al alumno inscribirse a una de las carreras activas. Un alumno solo puede tener una carrera en estado INSCRIPTO a la vez. Si ya está inscripto, el sistema redirige al home avisándolo.

Botones y redirecciones

- **Inscribirme** → Abre popup “¿Confirma la inscripción a la carrera?” → Confirmar: POST /alumno/carrera → /alumno/home.jsp (Inscripción realizada)
- **Cancelar (en popup)** → cierra el popup
- **Volver** → /alumno/home.jsp

Cambios de estado que genera la acción

- Inserta (o reactiva con ON DUPLICATE KEY) la fila en alumno_carrera con estado INSCRIPTO.

Reglas de negocio

- Solo se listan carreras activas.
- Si el alumno ya tiene una carrera INSCRIPTO, redirige al home con “Ya estás inscripto a la carrera X”.
- Si no hay carreras activas, el botón queda deshabilitado.

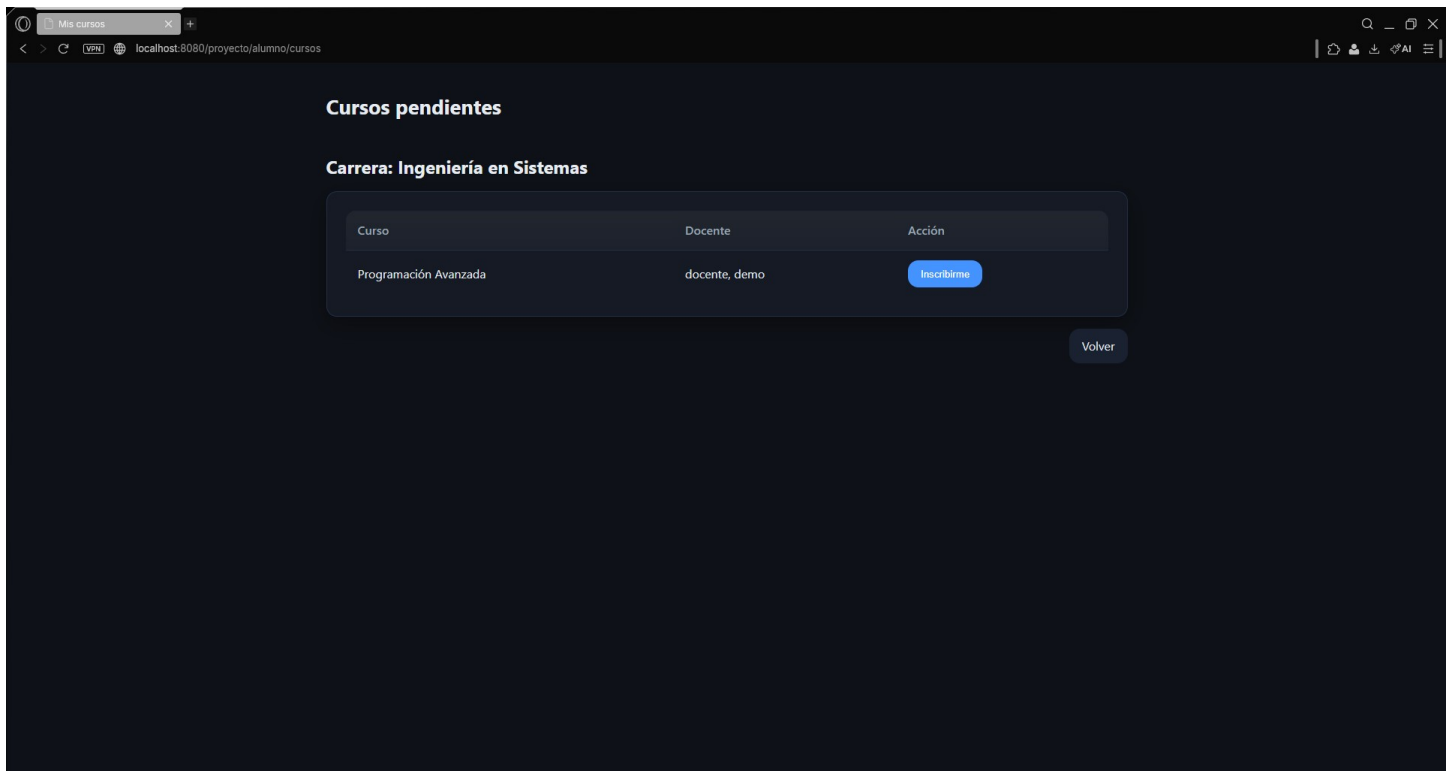
Validaciones y mensajes de error exactos

- No selecciona carrera: *“Seleccione una carrera”*
- Sin carreras activas: *“No hay carreras activas disponibles para inscripción.”*
- Error al guardar: *“Error al guardar la inscripción”*

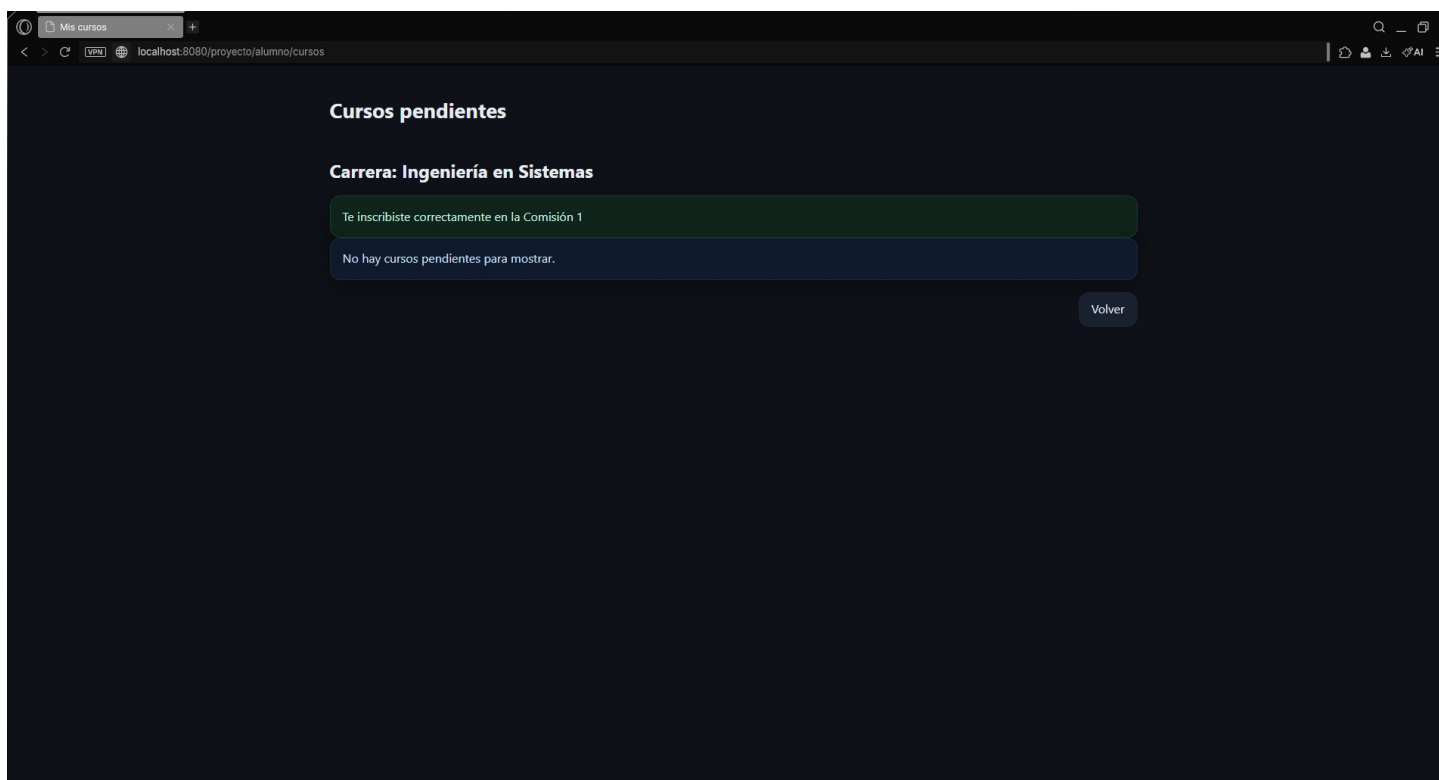
Mensajes de éxito (OK)

- *Al inscribirse redirige al Home Alumno con «Inscripción realizada».*
- *Si ya estaba inscripto: «Ya estás inscripto a la carrera X».*

22. Inscribir curso



Cursos pendientes de la carrera.



Inscripción confirmada en la Comisión 1.

Servlet / JSP: AlumnoInscribirCurso (GET/POST) – rol ALUMNO

Descripción funcional

Lista los cursos activos de la carrera del alumno que aún no cursó. Al inscribirse, el sistema asigna automáticamente la primera comisión con cupo disponible.

Columnas de la tabla: Curso, Docente, Acción (Inscribirme)

Botones y redirecciones

- **Inscribirme** → POST /alumno/cursos → éxito: /alumno/cursos?ok=Te+inscribiste+correctamente+en+la+Comisión+N
- **Volver** → /alumno/home.jsp

Cambios de estado que genera la acción

- Inserta una fila en comision_alumno con estado “cursando” en la comisión asignada.

Reglas de negocio

- Solo aparecen cursos activos de la carrera del alumno en los que aún no está inscripto.
- La comisión se asigna automáticamente: la primera (por orden) con cupo disponible.

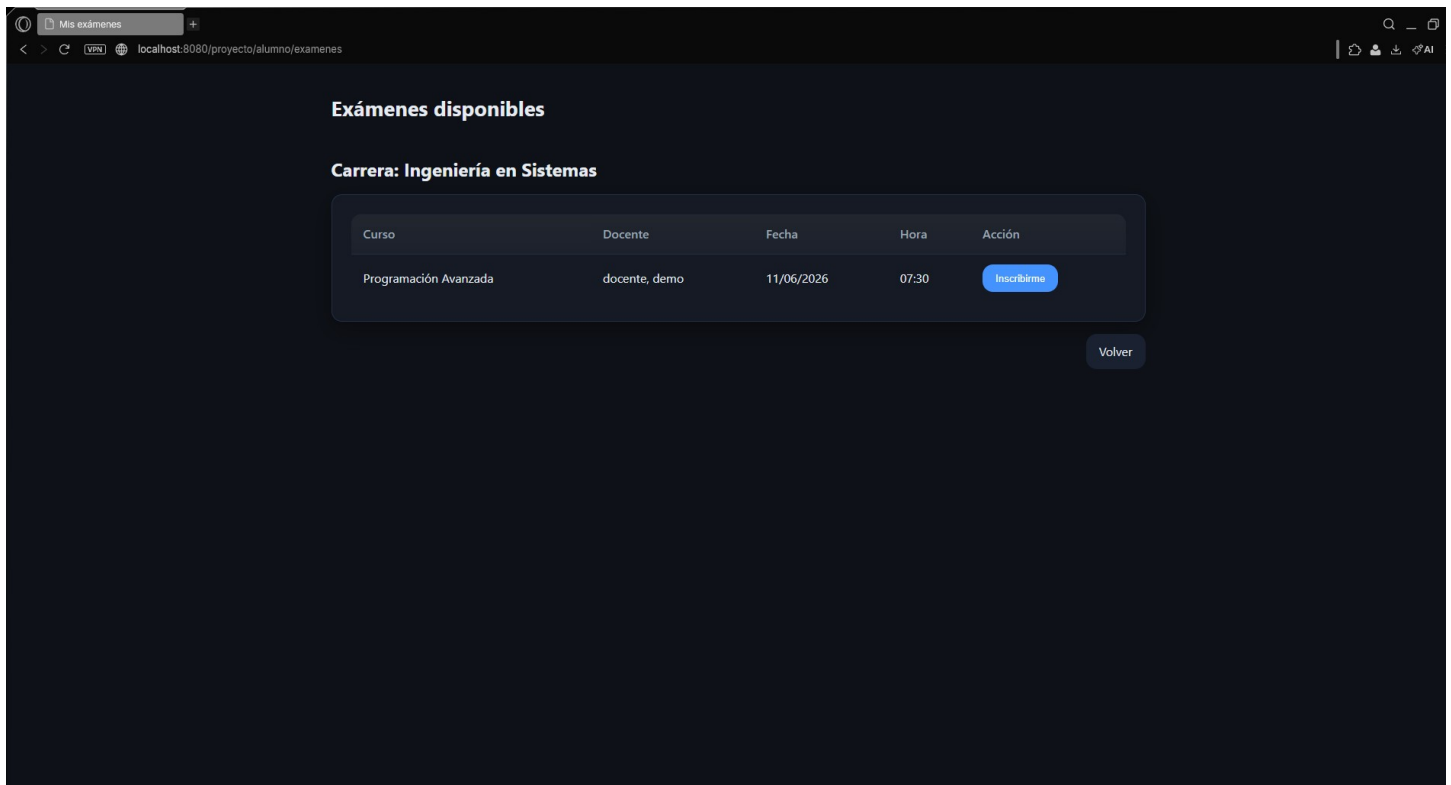
Validaciones y mensajes de error exactos

- Curso de otra carrera: *“El curso no pertenece a tu carrera”*
- Ya inscripto en el curso: *“Ya estás inscripto en este curso”*
- Sin cupo: *“No hay cupo disponible en ninguna comisión”*
- Solicitud inválida: *“Solicitud inválida”*
- Curso inválido: *“Curso inválido”*
- Error general: *“No se pudo realizar la inscripción”*
- Sin cursos pendientes: *“No hay cursos pendientes para mostrar.”*

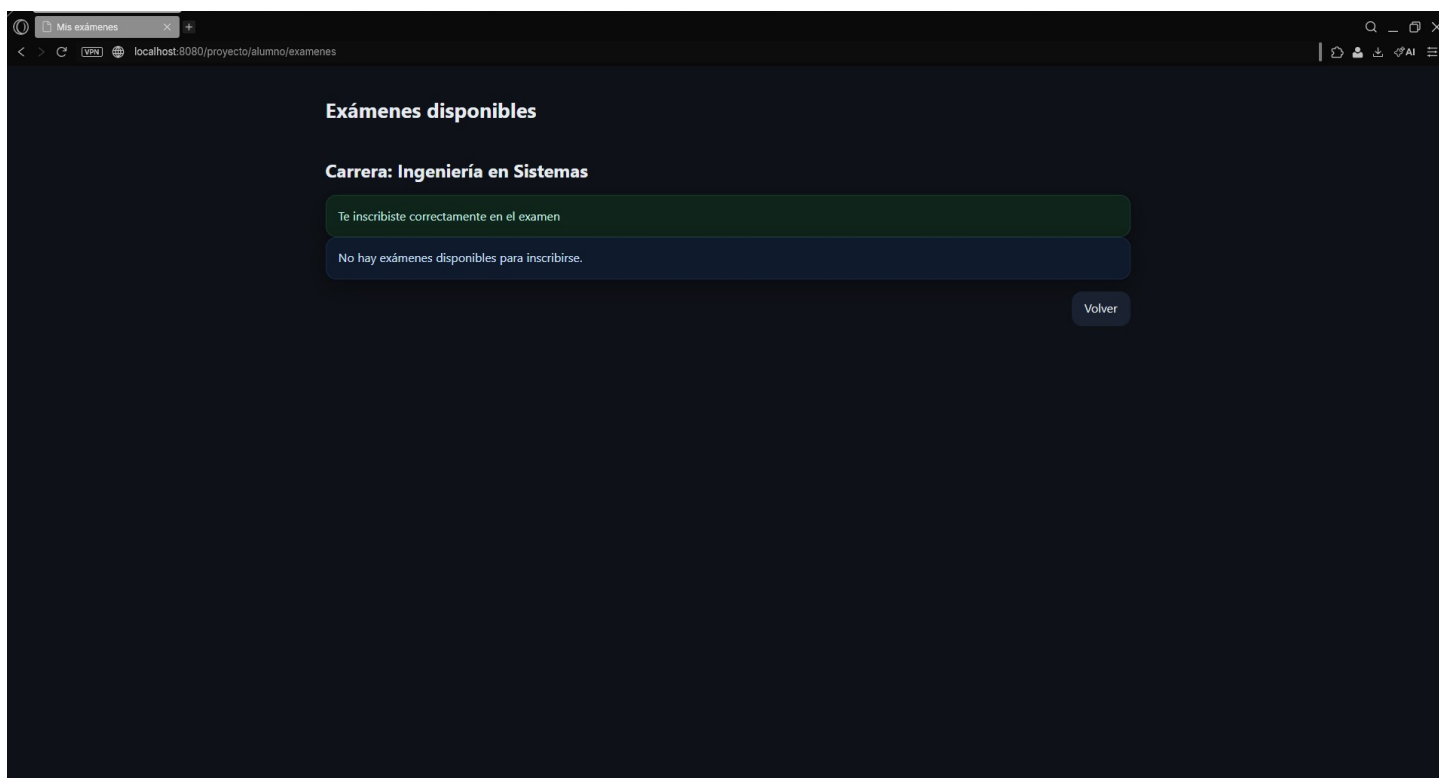
Mensajes de éxito (OK)

- «Te inscribiste correctamente en la Comisión N» (N = número de comisión asignada).

23. Inscribir examen



Exámenes disponibles.



Inscripción confirmada en el examen.

Servlet / JSP: AlumnoInscribirExamen (GET/POST) – rol ALUMNO

Descripción funcional

Lista los exámenes disponibles para el alumno: activos, con más de 24 horas de anticipación, de comisiones en las que el alumno quedó regular o libre y a los que aún no se inscribió.

Columnas de la tabla: Curso, Docente, Fecha, Hora, Acción (Inscribirme)

Botones y redirecciones

- **Inscribirme** → POST /alumno/examenes → éxito: /alumno/examenes?ok=Te+inscribiste+correctamente+en+el+examen
- **Volver** → /alumno/home.jsp

Cambios de estado que genera la acción

- Inserta una fila en examen_alumno (asistencia inicial “pendiente”, nota nula).

Reglas de negocio

- Solo exámenes activos, de comisiones donde el alumno está “regular” o “libre”.
- Debe faltar más de 24 horas para el examen.
- El alumno no debe estar ya inscripto en ese examen.

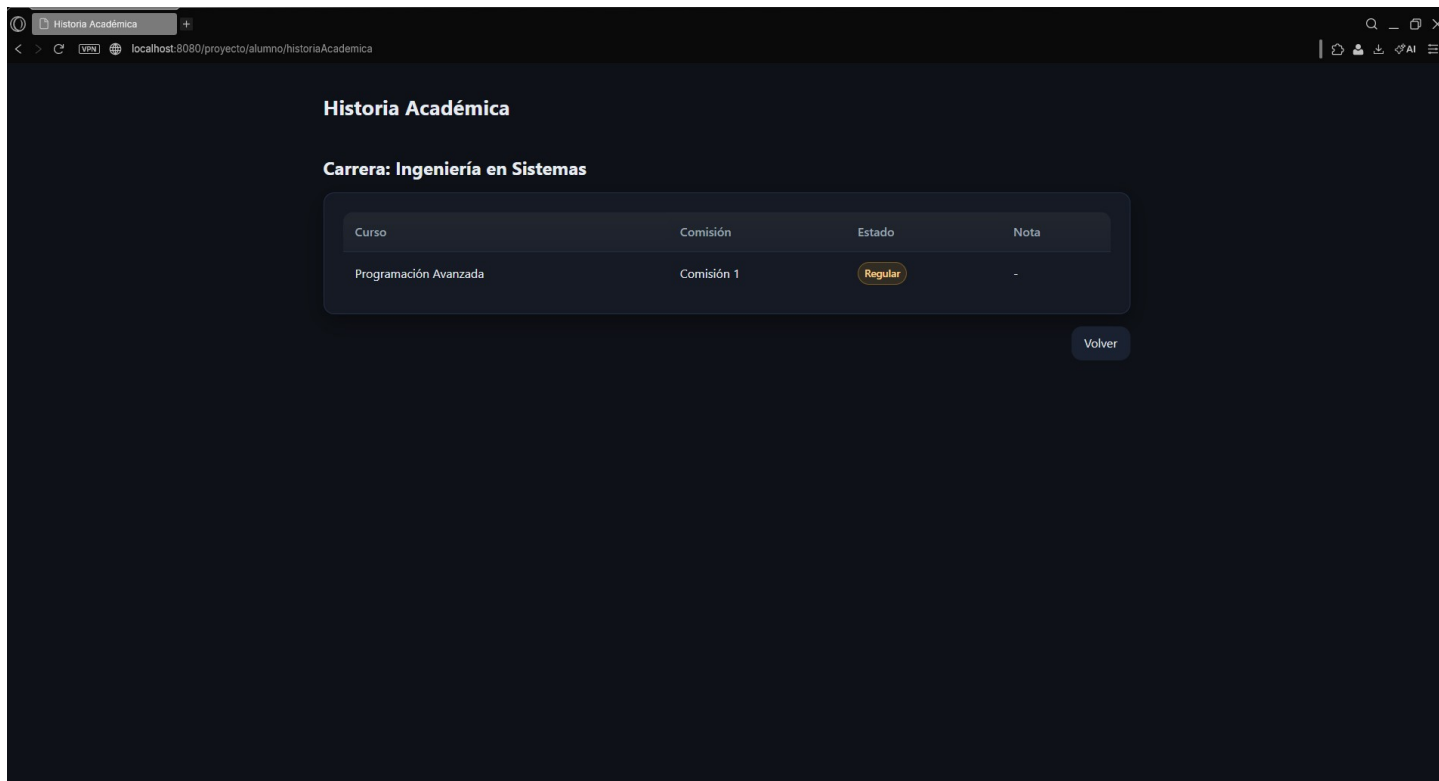
Validaciones y mensajes de error exactos

- Examen no disponible / <24hs: *“El examen no está disponible”*
- Ya inscripto en el examen: *“Ya estás inscripto en este examen”*
- Solicitud inválida: *“Solicitud inválida”*
- Examen inválido: *“Examen inválido”*
- Error general: *“No se pudo realizar la inscripción”*
- Sin exámenes disponibles: *“No hay exámenes disponibles para inscribirse.”*

Mensajes de éxito (OK)

- *«Te inscribiste correctamente en el examen».*

24. Historia académica (estado actual)



Servlet / JSP: AlumnoHistoriaAcademica (GET) – rol ALUMNO

Descripción funcional

Muestra los cursos cursados/en curso del alumno con su comisión, estado y nota. En esta instancia el curso aparece como “Regular” (el examen aún no fue rendido).

Columnas de la tabla: Curso, Comisión, Estado, Nota

Botones y redirecciones

- **Volver** → /alumno/home.jsp

Reglas de negocio

- El estado se muestra con badge de color: Promocionado (verde), Regular (amarillo), Cursando (azul), Libre (rojo), Aprobado (verde).

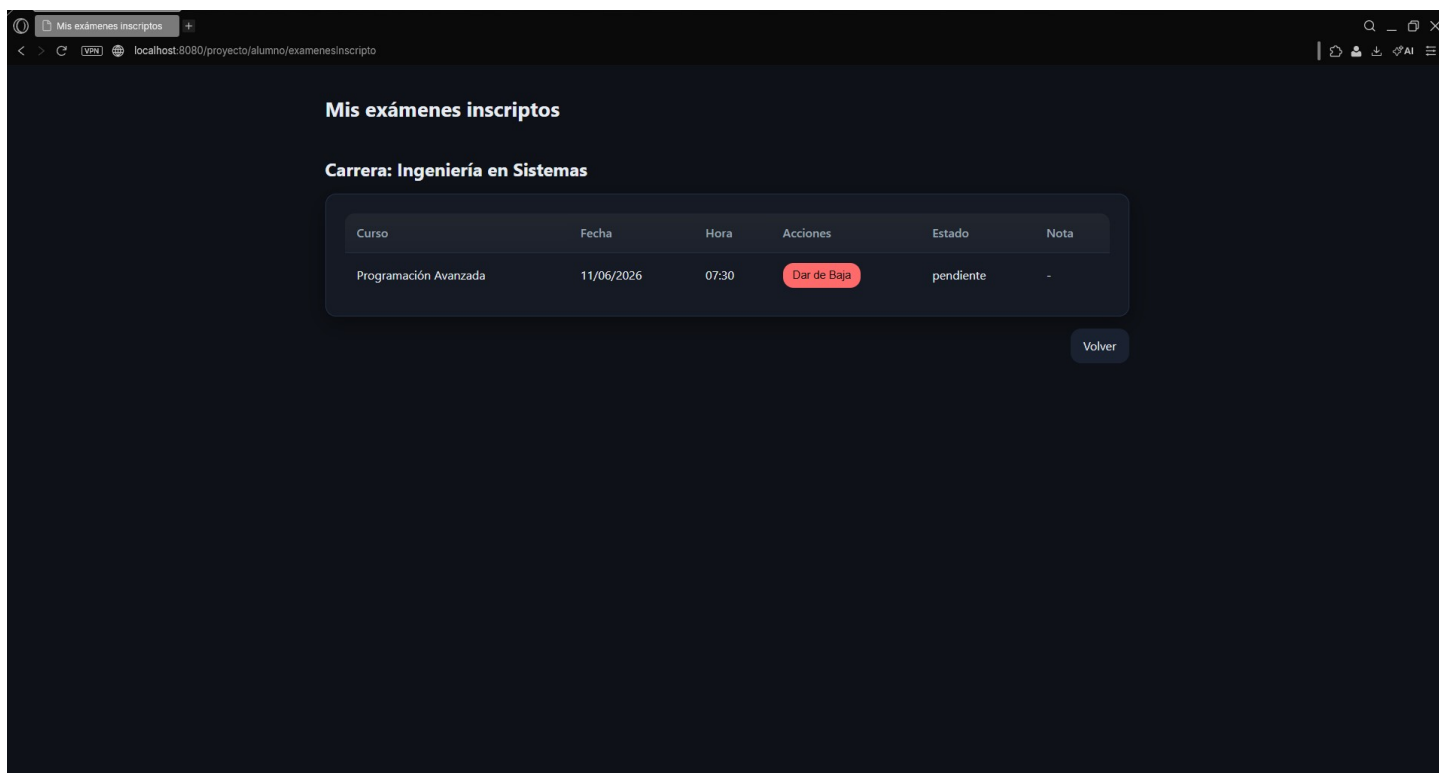
Validaciones y mensajes de error exactos

- Sin cursos: *“No hay cursos en tu historia académica.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *No muestra mensajes de éxito (vista de solo lectura).*

25. Mis exámenes (estado actual)



Servlet / JSP: AlumnoExamenInscripto (GET) – rol ALUMNO

Descripción funcional

Lista los exámenes a los que el alumno está inscripto, con su asistencia y nota. Permite darse de baja del examen solo si faltan más de 24 horas. En esta captura el examen figura “pendiente” y habilita “Dar de Baja”.

Columnas de la tabla: Curso, Fecha, Hora, Acciones (Dar de Baja), Estado (asistencia), Nota

Botones y redirecciones

- **Dar de Baja (si faltan >24hs)** → Abre popup “¿Confirma la baja del examen?” → Confirmar: POST /alumno/bajaExamen → /alumno/examenesInscripto?ok=Baja+realizada+correctamente
- **Volver** → /alumno/home.jsp

Cambios de estado que genera la acción

- La baja elimina la fila en examen_alumno (DELETE).

Reglas de negocio

- El botón “Dar de Baja” solo se muestra si faltan más de 24 horas para el examen.

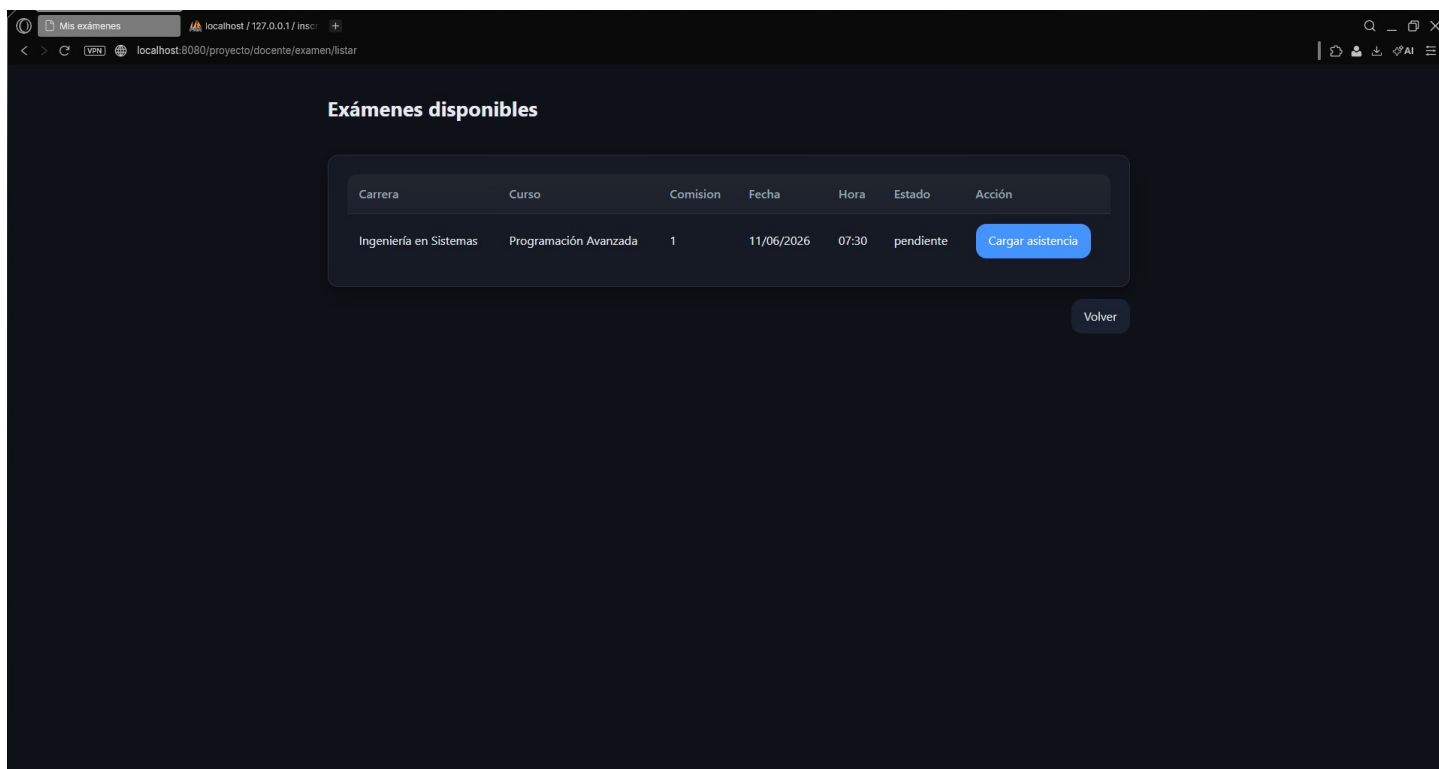
Validaciones y mensajes de error exactos

- Baja con <24hs: *“No se puede dar de baja el día anterior o el mismo día del examen”*
- Inscripción inexistente: *“No se encontró la inscripción”*
- Datos inválidos: *“Datos inválidos”*
- Error general: *“Error al dar de baja”*
- Sin exámenes inscriptos: *“No hay exámenes inscripto.”*

Mensajes de éxito (OK)

- *«Baja realizada correctamente» (al dar de baja un examen).*

26. Listar exámenes (docente)



Servlet / JSP: DocenteListarExamen (GET) – rol DOCENTE

Descripción funcional

Lista los exámenes activos del docente. La acción ofrecida depende del estado del examen y de si su fecha ya pasó. En esta captura el examen está “pendiente” y su fecha ya llegó, por lo que habilita “Cargar asistencia”.

Columnas de la tabla: Carrera, Curso, Comisión, Fecha, Hora, Estado, Acción

Botones y redirecciones

- **Cargar asistencia (Pendiente + fecha pasada)** → /docente/examen/gestion?idExamen=...&accion=Asistencia
- **Pendiente de fecha (Pendiente + fecha futura)** → texto informativo, sin acción
- **Cargar notas (Rendido)** → /docente/examen/gestion?idExamen=...&accion=Nota
- **Ver resultado (Cerrado)** → /docente/examen/gestion?idExamen=...&accion=Resultado
- **Volver** → /docente/home.jsp

Validaciones y mensajes de error exactos

- Sin exámenes asignados: *“No tenés exámenes asignados.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *«Guardado correctamente» (tras cargar asistencia o notas).*

27. Cargar asistencia

Alumnos inscriptos en examen

DNI	Nombre	Asistencia
2	Alumno, Demo	Seleccionar
3	demo, alumno2	Seleccionar Presente Ausente

[Confirmar Asistencia](#) [Volver](#)

Servlet / JSP: DocenteGestionExamen (GET/POST, accion=asistencia) – rol DOCENTE

Descripción funcional

Formulario para registrar la asistencia (Presente / Ausente) de cada alumno inscripto al examen. Solo puede usarse una vez que el examen ya ocurrió. Al confirmar, el examen pasa a estado “rendido”.

Columnas de la tabla: DNI, Nombre, Asistencia (Presente / Ausente)

Botones y redirecciones

- **Confirmar Asistencia** → Abre popup “¿Cargar los datos? Esta acción no se puede editar.” → Confirmar: POST → /docente/examen/listar?ok=Guardado+correctamente
- **Cancelar (en popup)** → cierra el popup
- **Volver** → /docente/examen/listar

Cambios de estado que genera la acción

- examen.estado pasa de “pendiente” a “rendido”; examen_alumno.asistencia se setea en presente/ausente.

Reglas de negocio

- Solo se permite cargar asistencia si la fecha y hora del examen ya pasaron.

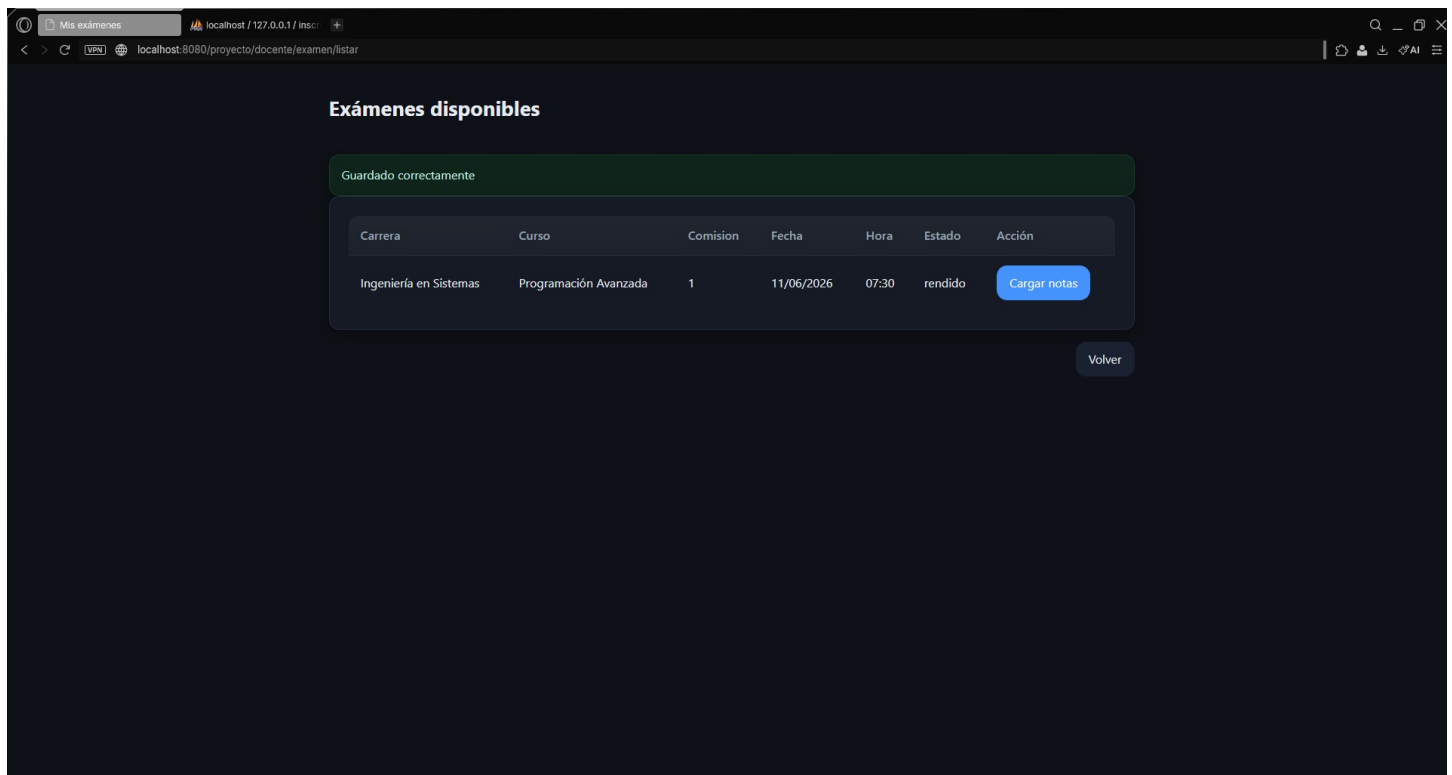
Validaciones y mensajes de error exactos

- Cargar antes del examen: *“No se puede cargar asistencia antes de que ocurra el examen”*
- Valor de asistencia inválido: *“asistencia invalida para el alumno con dni X”*
- Sin alumnos inscriptos: *“No hay alumnos inscriptos”*
- Error al guardar: *“Error al guardar”*
- Error de validación de fecha: *“Error al validar la fecha del examen”*

Mensajes de éxito (OK)

- *Al confirmar redirige a Mis exámenes (docente) con «Guardado correctamente».*

28. Listar exámenes (docente) – luego de asistencia



Servlet / JSP: DocenteListarExamen (GET) – rol DOCENTE

Descripción funcional

Tras cargar la asistencia, el listado muestra el mensaje “Guardado correctamente”, el examen cambia a estado “rendido” y la acción disponible pasa a ser “Cargar notas”.

Columnas de la tabla: Carrera, Curso, Comisión, Fecha, Hora, Estado (rendido), Acción (Cargar notas)

Botones y redirecciones

- **Cargar notas** → /docente/examen/gestion?idExamen=...&accion=Nota
- **Volver** → /docente/home.jsp

Cambios de estado que genera la acción

- Refleja el examen ya en estado “rendido”.

Mensajes de éxito (OK)

- *«Guardado correctamente».*

29. Cargar notas

DNI	Nombre	Nota
3	demo, alumno2	1-10

[Confirmar Nota](#) [Volver](#)

Servlet / JSP: DocenteGestionExamen (GET/POST, accion=nota) – rol DOCENTE

Descripción funcional

Formulario para cargar la nota (1 a 10) de los alumnos que estuvieron presentes (los ausentes no aparecen). Al confirmar, el examen pasa a “cerrado”; los alumnos con $\text{nota} \geq 6$ se marcan como “aprobado” en su comisión y se verifica si completaron la carrera.

Columnas de la tabla: DNI, Nombre, Nota (1-10)

Botones y redirecciones

- **Confirmar Nota** → Abre popup “¿Cargar los datos? Esta acción no se puede editar.” → Confirmar: POST → /docente/examen/listar?ok=Guardado+correctamente
- **Cancelar (en popup)** → cierra el popup
- **Volver** → /docente/examen/listar

Cambios de estado que genera la acción

- examen.estado pasa de “rendido” a “cerrado”; examen_alumno.nota se setea.
- Alumnos con $\text{nota} \geq 6$ → comision_alumno.estado = “aprobado” (con su nota).
- Si con ese aprobado el alumno completó todos los cursos de la carrera, su alumno_carrera pasa a APROBADO.

Reglas de negocio

- Solo se procesan los alumnos con asistencia “presente”.
- La nota debe estar entre 1 y 10.

Validaciones y mensajes de error exactos

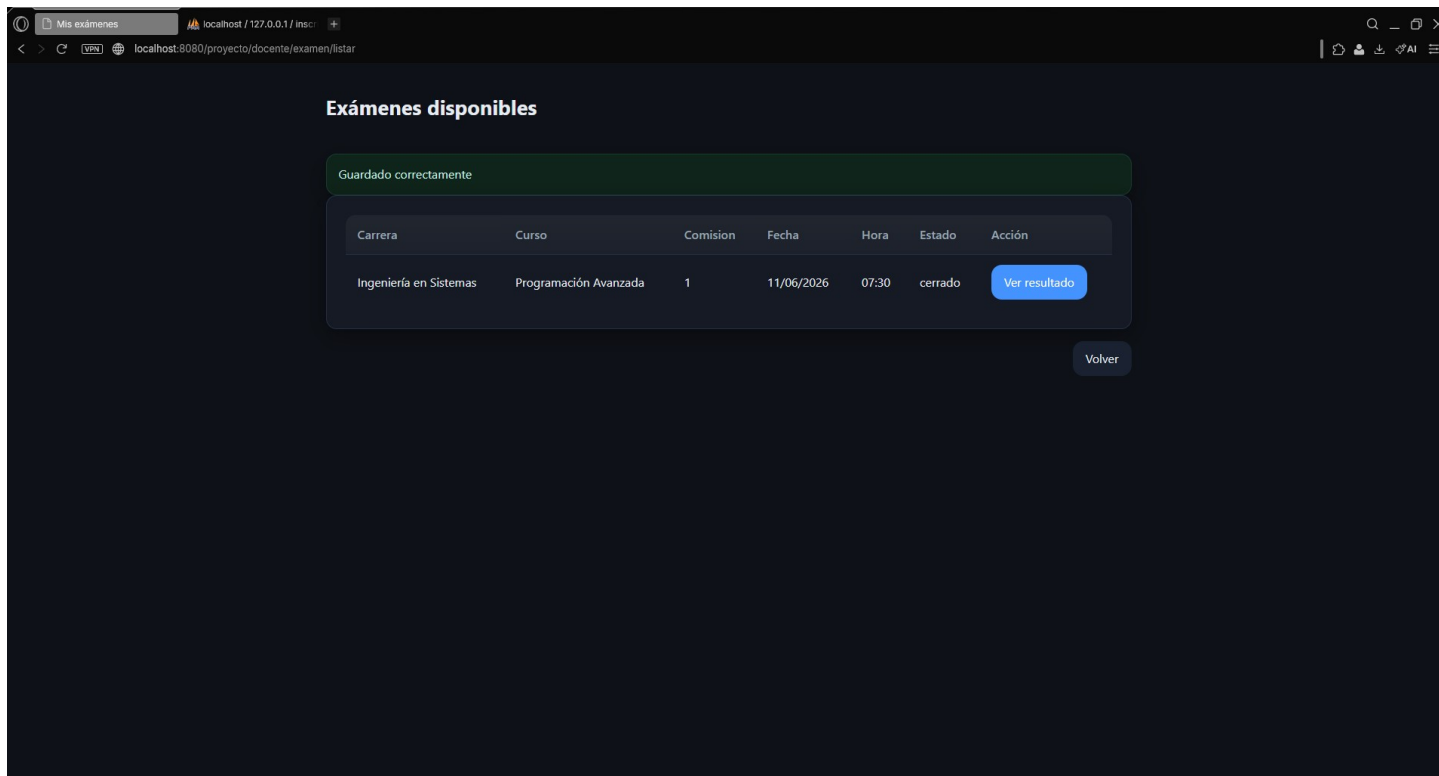
- Nota inválida: *“nota invalida para el alumno con dni X”*
- Sin alumnos inscriptos: *“No hay alumnos inscriptos”*

- Error al guardar: *“Error al guardar”*

Mensajes de éxito (OK)

- *Al confirmar redirige a Mis exámenes (docente) con «Guardado correctamente».*

30. Listar exámenes (docente) – luego de notas



Servlet / JSP: DocenteListarExamen (GET) – rol DOCENTE

Descripción funcional

Tras cargar las notas, el listado muestra “Guardado correctamente”, el examen queda en estado “cerrado” y la única acción disponible es “Ver resultado”.

Columnas de la tabla: Carrera, Curso, Comisión, Fecha, Hora, Estado (cerrado), Acción (Ver resultado)

Botones y redirecciones

- **Ver resultado** → /docente/examen/gestion?idExamen=...&accion=Resultado
- **Volver** → /docente/home.jsp

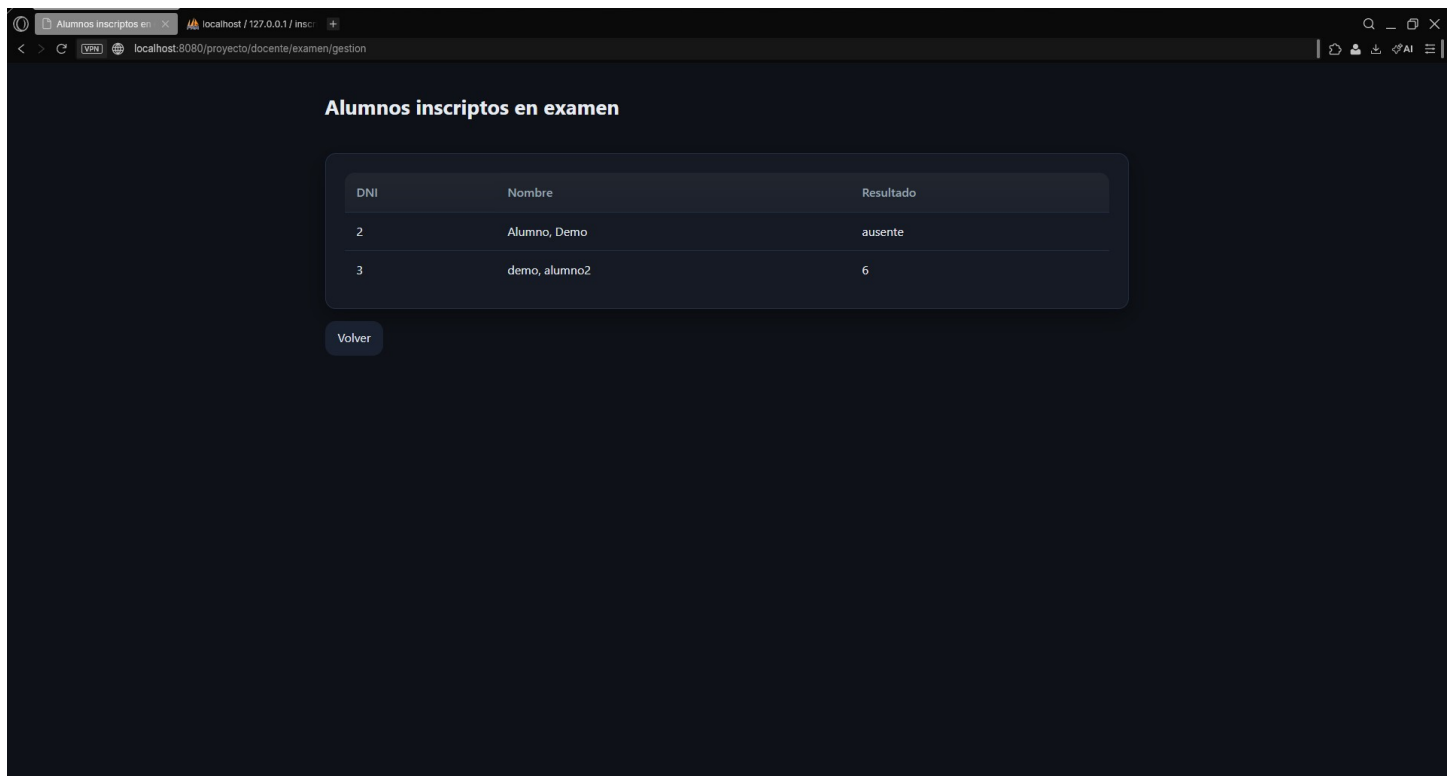
Cambios de estado que genera la acción

- Refleja el examen ya en estado “cerrado”.

Mensajes de éxito (OK)

- *«Guardado correctamente».*

31. Ver resultado



Servlet / JSP: DocenteGestionExamen (GET, accion=resultado) – rol DOCENTE

Descripción funcional

Vista de solo lectura del resultado del examen. Para cada alumno muestra la nota si estuvo presente, o “ausente” si no asistió. No tiene botón de confirmación.

Columnas de la tabla: DNI, Nombre, Resultado (nota o “ausente”)

Botones y redirecciones

- **Volver** → /docente/examen/listar

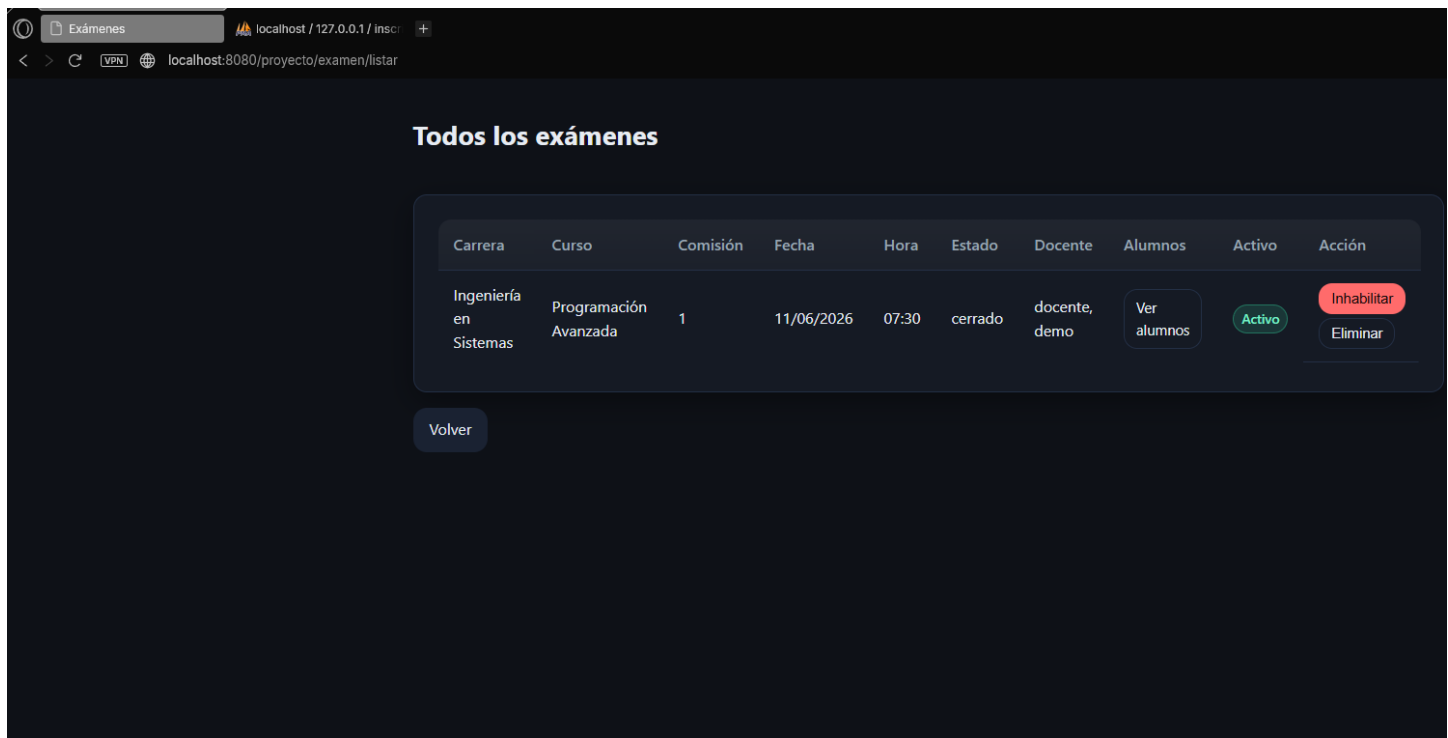
Mensajes de éxito (OK)

- *No muestra mensajes de éxito (vista de solo lectura).*

Notas

- En la captura: DNI 2 (Alumno Demo) figura “ausente” y DNI 3 (alumno2 demo) con nota 6.

32. Listar exámenes (admin) – estado cerrado



Servlet / JSP: AdminListarExamen (GET) – rol ADMIN

Descripción funcional

El mismo listado de exámenes del administrador, ahora reflejando el examen en estado “cerrado” tras completarse el ciclo de asistencia y notas.

Columnas de la tabla: Carrera, Curso, Comisión, Fecha, Hora, Estado (cerrado), Docente, Alumnos, Activo, Acción

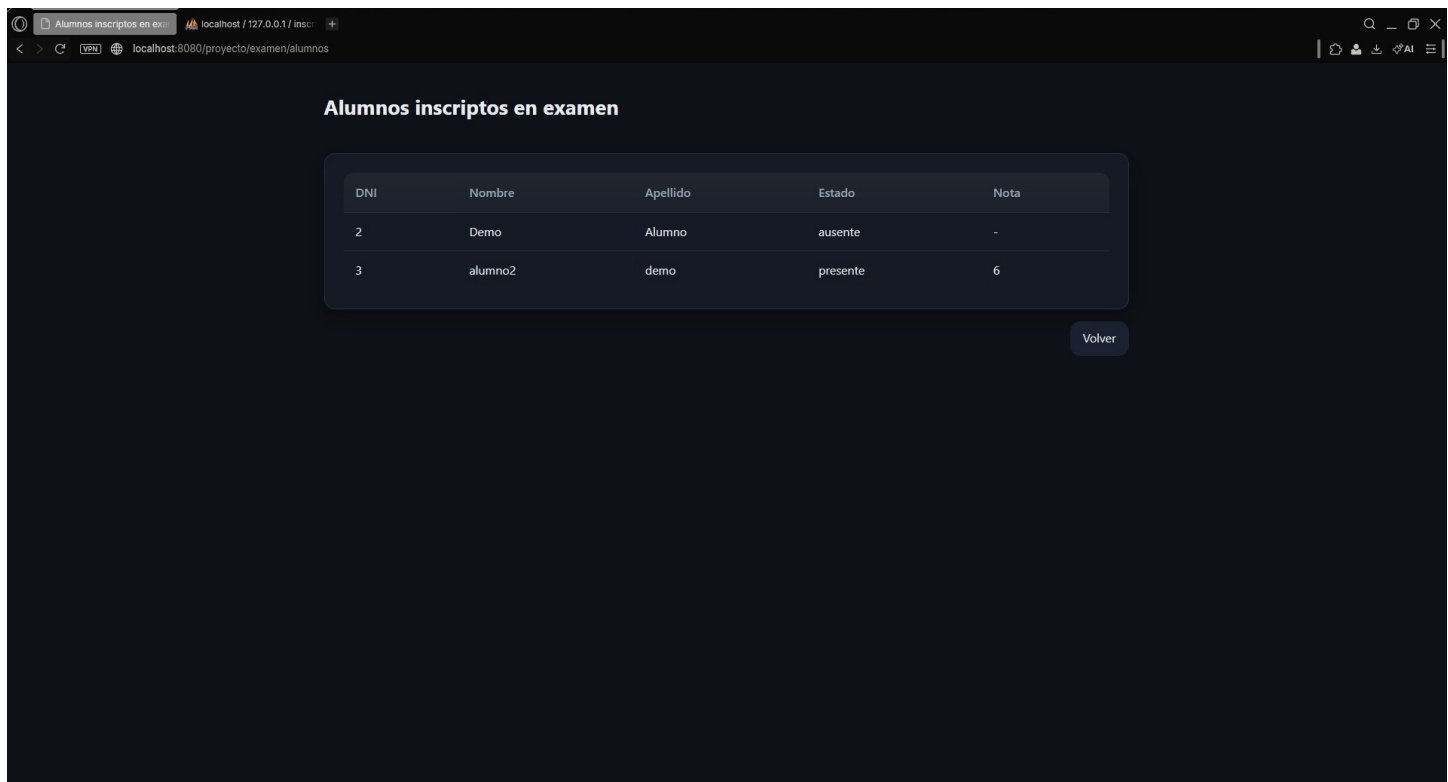
Botones y redirecciones

- **Ver alumnos** → /examen/alumnos?idExamen=
- **Inhabilitar / Eliminar** → con popup (ver pantalla 16)
- **Volver** → /admin/home.jsp

Mensajes de éxito (OK)

- *Mismos mensajes que la pantalla 16: «Exámen creado correctamente», «examen habilitado/a / inhabilitado/a correctamente», «examen eliminado/a correctamente».*

33. Alumnos del examen (admin)



Alumnos inscriptos en examen

DNI	Nombre	Apellido	Estado	Nota
2	Demo	Alumno	ausente	-
3	alumno2	demo	presente	6

Volver

Servlet / JSP: AdminListarAlumnoExamen (GET) – rol ADMIN

Descripción funcional

Vista del administrador con los alumnos inscriptos a un examen, su asistencia y su nota. En la captura: DNI 2 ausente (sin nota) y DNI 3 presente con nota 6.

Columnas de la tabla: DNI, Nombre, Apellido, Estado (asistencia), Nota

Botones y redirecciones

- **Volver** → /examen/listar?idComision=

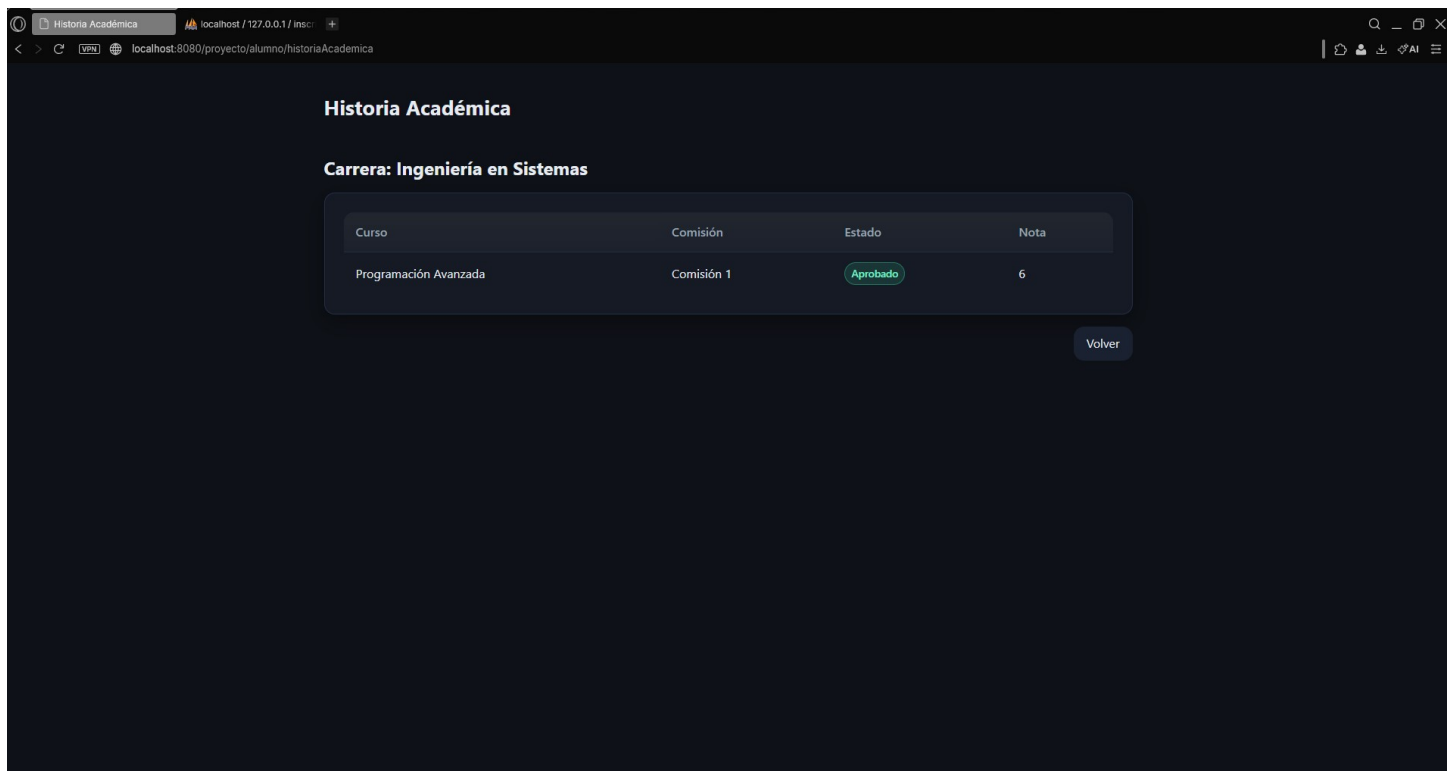
Validaciones y mensajes de error exactos

- Examen sin alumnos: *“No hay alumnos inscriptos en este examen.”*
- Error de base de datos: *“Error al acceder a la base de datos”*

Mensajes de éxito (OK)

- *No muestra mensajes de éxito.*

34. Historia académica (actualizada)



Servlet / JSP: AlumnoHistoriaAcademica (GET) – rol ALUMNO

Descripción funcional

La historia académica del alumno luego de aprobar el examen: el curso figura ahora como “Aprobado” con nota 6 (cambio de estado disparado por la carga de notas del docente).

Columnas de la tabla: Curso, Comisión, Estado (Aprobado), Nota (6)

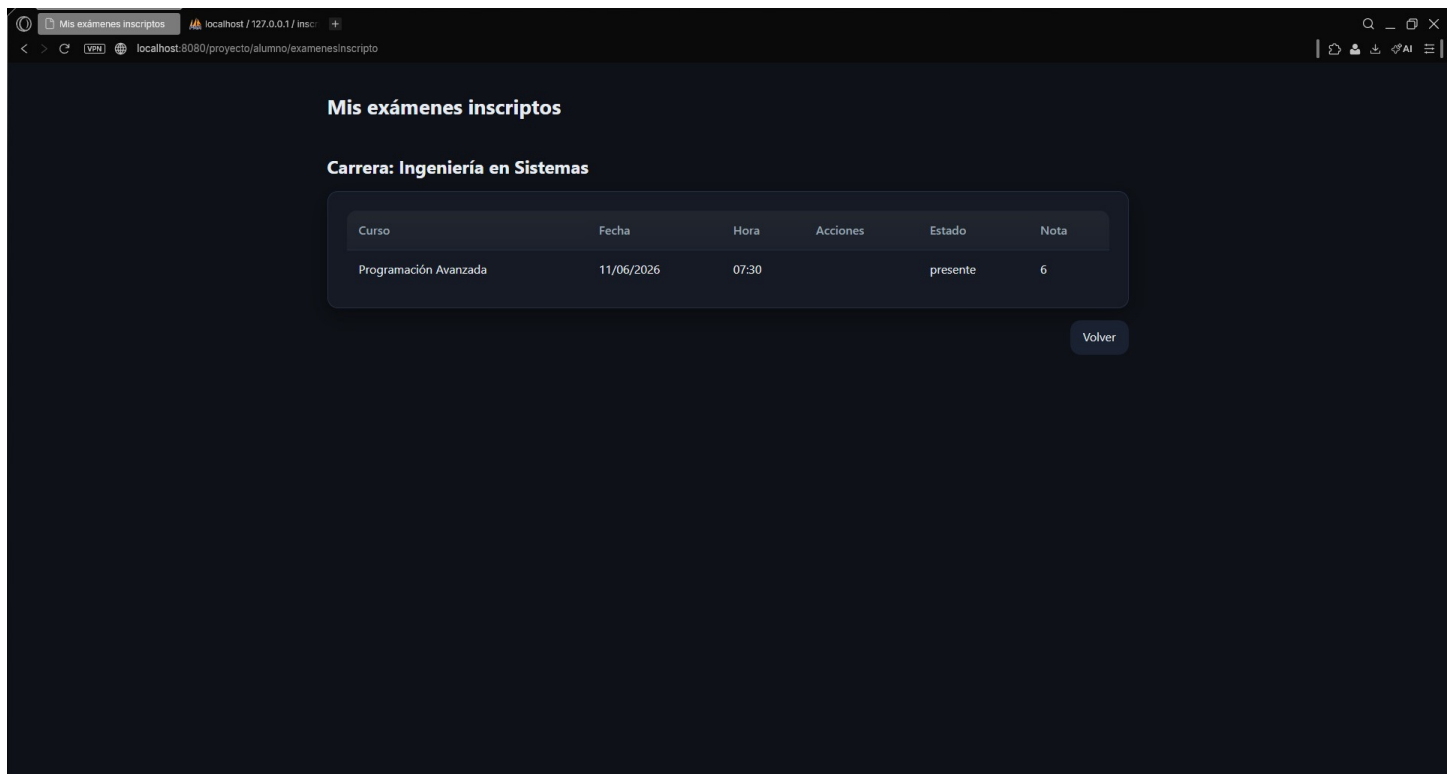
Botones y redirecciones

- **Volver** → /alumno/home.jsp

Mensajes de éxito (OK)

- *No muestra mensajes de éxito (vista de solo lectura).*

35. Mis exámenes (actualizado)



Servlet / JSP: AlumnoExamenInscripto (GET) – rol ALUMNO

Descripción funcional

La lista de exámenes inscriptos del alumno tras rendir: muestra asistencia “presente” y nota 6. Ya no aparece el botón “Dar de Baja” porque la fecha del examen pasó.

Columnas de la tabla: Curso, Fecha, Hora, Acciones, Estado (presente), Nota (6)

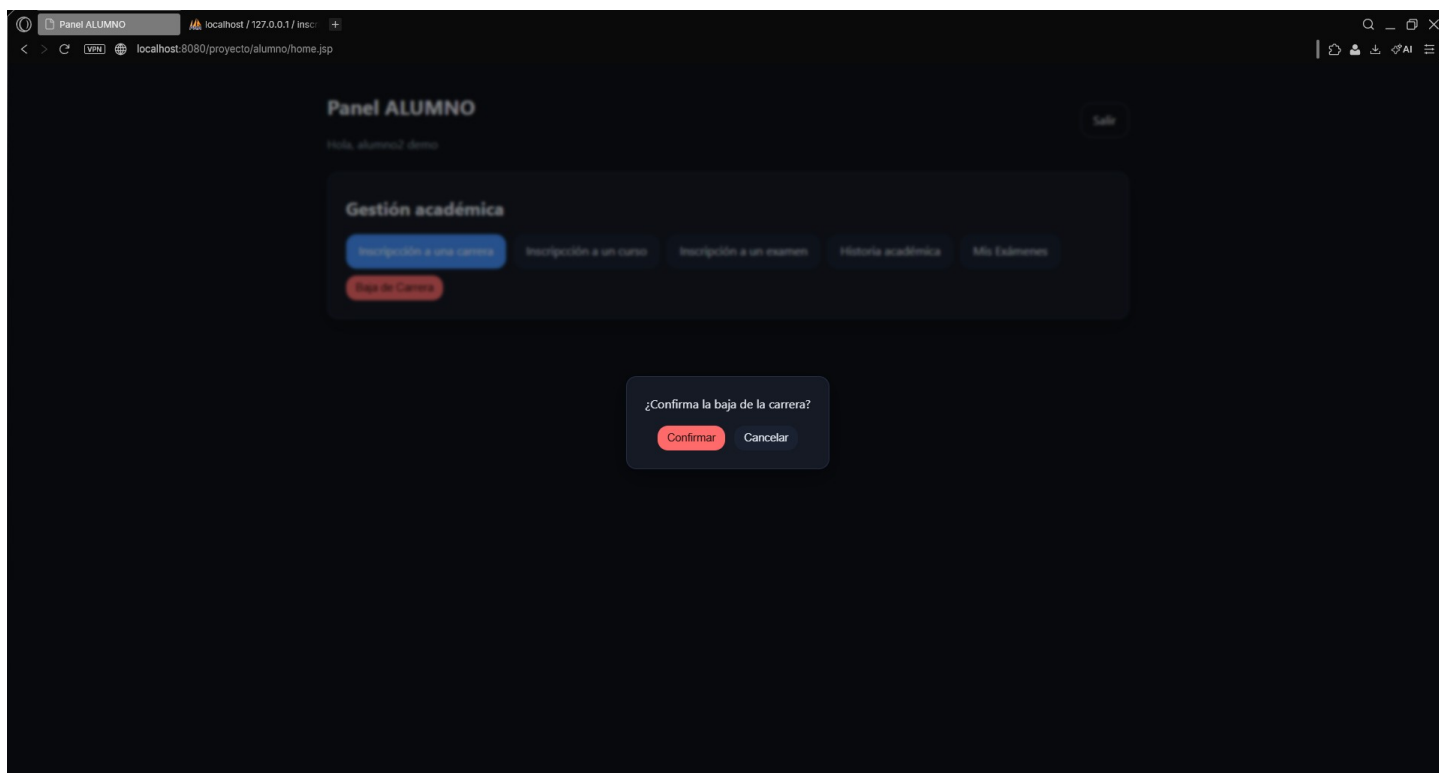
Botones y redirecciones

- **Volver** → /alumno/home.jsp

Mensajes de éxito (OK)

- «Baja realizada correctamente» (cuando la baja del examen es posible, > 24 hs).

36. Popup de confirmación – baja de carrera



Servlet / JSP: alumno/home.jsp (popover HTML nativo) + AlumnoBajaCarrera (POST)

Descripción funcional

Al presionar “Baja de Carrera” en el Home Alumno se abre un popover nativo de confirmación. El botón Confirmar envía el formulario de baja; Cancelar cierra el popover sin acción. El mensaje exacto del popup es “¿Confirma la baja de la carrera?”.

Botones y redirecciones

- **Confirmar** → POST /alumno/carrera/baja → Home Alumno con «Baja realizada»
- **Cancelar** → cierra el popover, permanece en /alumno/home.jsp

Cambios de estado que genera la acción

- UPDATE alumno_carrera SET estado = 'BAJA' para la carrera INSCRIPTO del alumno.

Reglas de negocio

- El sistema usa el atributo HTML nativo popover para las confirmaciones, sin JavaScript: un botón abre el overlay y un botón Confirmar con el atributo form= envía el formulario asociado.

Validaciones y mensajes de error exactos

- Sin carrera inscripta: *“No estás inscripto en ninguna carrera.”*
- Error general: *“No se pudo gestionar la baja”*

Mensajes de éxito (OK)

- *Al confirmar redirige al Home Alumno con «Baja realizada».*

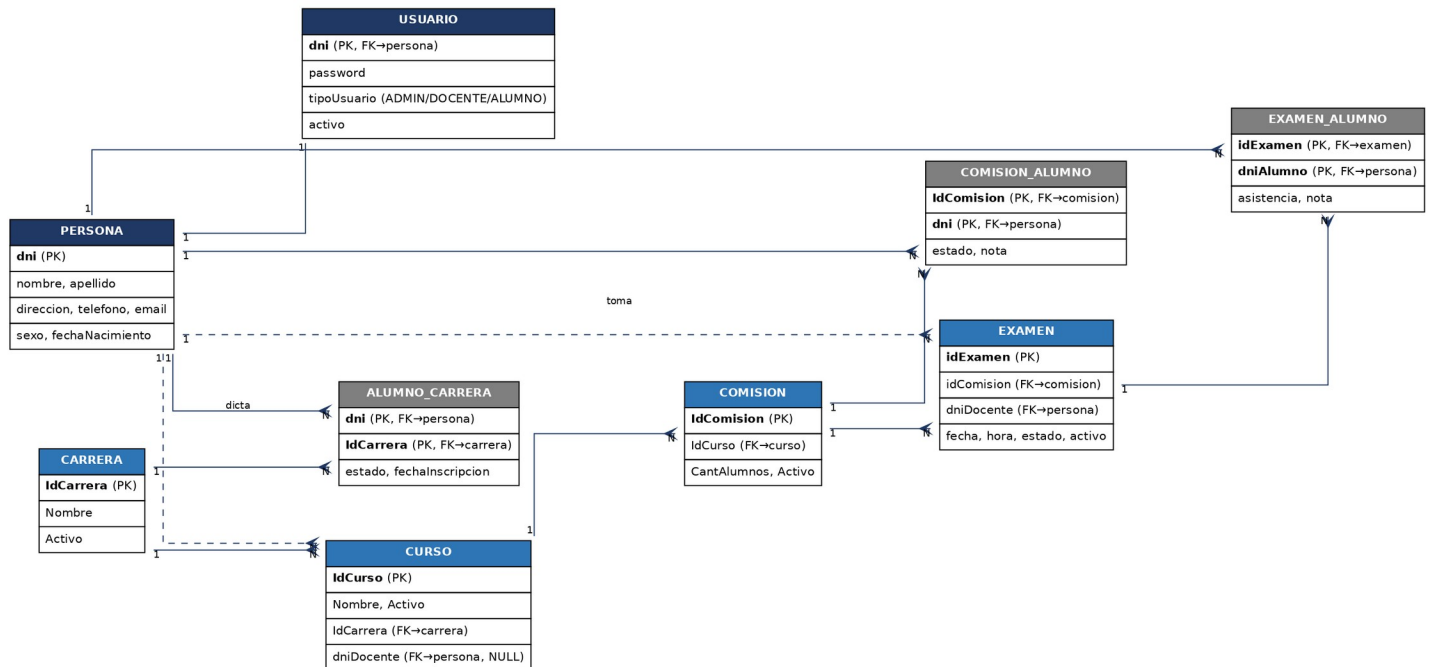
Catálogo de acciones con popup de confirmación

Complemento del punto 36. El sistema utiliza popovers HTML nativos de confirmación en las siguientes acciones. Para cada una se indica el mensaje exacto del popup y la pantalla resultante tras confirmar.

Acción	Mensaje del popup	Pantalla resultante tras confirmar
Inhabilitar carrera	¿Inhabilitar esta carrera?	Listar carreras (/carrera/listar)
Habilitar carrera	¿Habilitar esta carrera?	Listar carreras (/carrera/listar)
Eliminar carrera	¿Eliminar esta carrera?	Listar carreras (/carrera/listar)
Inhabilitar curso	¿Inhabilitar este curso?	Listar cursos (/curso/listar?idCarrera=)
Habilitar curso	¿Habilitar este curso?	Listar cursos (/curso/listar?idCarrera=)
Eliminar curso	¿Eliminar este curso?	Listar cursos (/curso/listar?idCarrera=)
Inhabilitar comisión	¿Inhabilitar esta comisión?	Listar comisiones (/comision/listar?idCurso=)
Habilitar comisión	¿Habilitar esta comisión?	Listar comisiones (/comision/listar?idCurso=)
Inhabilitar examen	¿Inhabilitar este examen?	Listar exámenes (/examen/listar)
Habilitar examen	¿Habilitar este examen?	Listar exámenes (/examen/listar)
Eliminar examen	¿Eliminar este examen?	Listar exámenes (/examen/listar)
Inhabilitar alumno	¿Inhabilitar este alumno?	Listar alumnos (/alumno/listar)
Habilitar alumno	¿Habilitar este alumno?	Listar alumnos (/alumno/listar)
Eliminar alumno	¿Eliminar este alumno?	Listar alumnos (/alumno/listar)
Inhabilitar docente	¿Inhabilitar este docente?	Listar docentes (/docente/listar)
Habilitar docente	¿Habilitar este docente?	Listar docentes (/docente/listar)
Eliminar docente	¿Eliminar este docente?	Listar docentes (/docente/listar)
Cierre de cursado (docente)	¿Confirma el cierre del cursado? Esta acción no se puede editar.	Mis cursos (/docente/curso/listar) – Cursado cerrado correctamente
Cargar asistencia / notas (docente)	¿Cargar los datos? Esta acción no se puede editar.	Mis exámenes (/docente/examen/listar) – Guardado correctamente
Inscripción a carrera (alumno)	¿Confirma la inscripción a la carrera?	Home Alumno (/alumno/home.jsp) – Inscripción realizada
Baja de examen (alumno)	¿Confirma la baja del examen?	Mis exámenes inscriptos – Baja realizada correctamente
Baja de carrera (alumno)	¿Confirma la baja de la carrera?	Home Alumno (/alumno/home.jsp) – Baja realizada

3. Diagrama Entidad-Relación (DER)

El siguiente diagrama representa el esquema definitivo de la base de datos “inscripciones” (script SQL final del proyecto). Las tablas de relación (alumno_carrera, comision_alumno y examen_alumno) modelan las inscripciones del alumno; el atributo dni de PERSONA actúa como referencia tanto para alumnos como para docentes.



DER del sistema (entidades, claves y cardinalidades).

Entidades principales

- PERSONA (dni PK) – datos personales; usada tanto para alumnos como para docentes.
- USUARIO (dni PK/FK → persona) – credenciales, rol (tipoUsuario: admin/docente/alumno) y estado activo. Relación 1:1 con PERSONA.
- CARRERA (IdCarrera PK) – 1:N con CURSO.
- CURSO (IdCurso PK) – FK a CARRERA y FK opcional dniDocente → PERSONA (docente que dicta el curso).
- COMISION (IdComision PK) – FK a CURSO; cada curso se crea con dos comisiones (cupó CantAlumnos; el número de comisión no se almacena, se calcula por consulta).
- EXAMEN (idExamen PK) – FK a COMISION y FK dniDocente → PERSONA; fecha, hora, estado (pendiente/rendido/cerrado/cancelado) y activo.

Tablas de relación (inscripciones)

- ALUMNO_CARRERA (PK compuesta dni + IdCarrera) – inscripción del alumno a una carrera, con estado (INSCRIPTO/BAJA/APROBADO) y fecha.
- COMISION_ALUMNO (PK compuesta IdComision + dni) – cursado del alumno en una comisión, con estado (cursando/regular/libre/promocionado/aprobado) y nota.
- EXAMEN_ALUMNO (PK compuesta idExamen + dniAlumno) – inscripción del alumno a un examen, con asistencia (pendiente/presente/ausente) y nota.

Todas las claves foráneas se definen con ON DELETE CASCADE, lo que permite la eliminación en cascada (por ejemplo, al eliminar un curso se eliminan sus comisiones). El número de comisión se calcula en las consultas con ROW_NUMBER()/DENSE_RANK() en lugar de almacenarse en la tabla.